Asistente RAG reproducible sobre prensa y Bluesky con Milvus y modelos locales: arquitectura, evaluación y compuerta de dominio

Erika Valeria Rivera Jimenez, Julian Guillermo Carrillo Meneses,
Carlos Andres Garnica Salazar, Oscar Andres Prieto Moreno

Pontificia Universidad Javeriana, Bogotá, Colombia
riveraev@javeriana.edu.co, carrillojuliang@javeriana.edu.co,
carlos_garnicas@javeriana.edu.co, o-prieto@javeriana.edu.co

***Resumen*— La recuperación aumentada por generación (RAG) responde sobre un corpus sin reentrenar el modelo, pero su calidad depende de decisiones que rara vez se documentan juntas: limpieza, segmentación, codificador, índice, reordenamiento, prompt y control de abstención. Este trabajo presenta un asistente RAG reproducible sobre 169.248 documentos de prensa colombiana e internacional y Bluesky, 161.868 indexados y 618.286 fragmentos en Milvus, con modelos locales. La comparación de capas es exploratoria sobre el silver y usa intervalos pareados donde existen. La configuración final combina bge-m3, la unión de recuperación densa y BM25 con un bono de entidades del grafo, el reordenador bge-reranker-v2-m3, el prompt v3, la traducción de la consulta y la extracción de listas. Un clasificador de intención basado en reglas separa la charla de la búsqueda, una compuerta logística decide si la pregunta es del dominio y un detector de ambigüedad, opcional, propone las interpretaciones. En test alcanza 0,958 de exactitud, 0,840 en el reservado y 0,884 en el silver completo; RAGAS 0,668 y 0,623, con fidelidad 1,0 y 0,947. El mismo contrato se implementó sobre Solr 9 y se añadió observabilidad con Langfuse y un grafo de entidades en Neo4j. Una ampliación aleatoria de 30 preguntas baja la exactitud a 0,800 y mantiene RAGAS en 0,674: el test curado es más fácil. Se documentan las mejoras rechazadas (contexto padre, modelo 27B, expansión de consulta, verificación numérica y garantía de candidato), el coste de cada decisión y los límites del corpus.**

***Palabras clave*— recuperación aumentada por generación, Milvus, bge-m3, reordenamiento, evaluación de sistemas de recuperación, compuerta de dominio, observabilidad, GraphRAG.**

I. Introducción

Un sistema de recuperación aumentada por generación (RAG) responde una pregunta con fragmentos recuperados de una colección y un modelo de lenguaje que redacta a partir de ellos [1]. La promesa es atractiva: actualizar el conocimiento sin reentrenar y citar la evidencia. En la práctica, la calidad depende de una cadena de decisiones que se afectan entre sí. Una limpieza agresiva puede borrar la frase que responde y un chunking pequeño pierde contexto. Un codificador entrenado en otro dominio recupera vecinos plausibles pero inútiles. Un reordenador puede mejorar el orden y, al mismo tiempo, encarecer cada consulta. Un prompt permisivo invita a inventar; sin un control de abstención, el sistema responde con seguridad sobre temas que el corpus no cubre.

La literatura de RAG suele evaluar un componente a la vez. BEIR y MTEB miden recuperadores [2], [3]; los benchmarks multilingües como MIRACL comparan codificadores [4]; RAGAS, ARES, eRAG y RAGChecker evalúan la respuesta generada [5], [6], [7], [8]. Son piezas necesarias, pero no dicen cómo se comporta la cadena completa sobre un corpus real, con duplicados, truncamiento, ruido de rastreo y preguntas sin respuesta. Tampoco dicen cuánto cuesta cada mejora ni cuándo conviene rechazarla.

Este trabajo construye y evalúa esa cadena completa sobre un corpus de 169.248 documentos recolectados de prensa colombiana e internacional y Bluesky, 161.868 indexados y 618.286 fragmentos. El sistema corre en una estación con una GPU AMD de 16 GB y un servidor de la red local que aloja Milvus [9]; los modelos se sirven por Ollama [10]. No se usan servicios externos de generación ni de embeddings. La Tabla V resume la configuración final y la Sección IV justifica cada elección.

I-A. Preguntas de investigación

1. **Calidad del corpus.** ¿Qué problemas de limpieza, truncamiento, duplicados e idioma aparecen en un corpus de prensa y redes sociales, y cuánto afectan la recuperación?
2. **Recuperación.** ¿Qué combinación de codificador, índice, tamaño de contexto y reordenamiento maximiza la cobertura y el orden con un coste aceptable?
3. **Generación y abstención.** ¿Cómo se comportan distintos modelos locales sobre la misma evidencia y qué controles evitan respuestas sin respaldo?
4. **Evaluación.** ¿Qué acuerdo alcanzan los anotadores humanos y qué señales de evaluación automática son fiables cuando el juez también es un modelo local?

I-B. Aportes

1. Un pipeline reproducible de extremo a extremo, con Docker Compose, Milvus, Ollama, FastAPI y Streamlit, que se levanta con un comando y no depende de servicios de pago.
2. Un análisis exploratorio y una textometría del corpus que cuantifican el texto sin cierre terminal (32,7 %, un proxy de truncamiento), ruido de rastreo, duplicados

exactos y near-duplicates, y que justifican una limpieza conservadora.
3. Una comparación pareada de cuatro variantes de limpieza, dos estrategias de segmentación, tres codificadores, recuperación densa, dispersa e híbrida, y tres alcances de reordenamiento, con intervalos de confianza por bootstrap.
4. Un silver standard de 43 preguntas con revisión de cuatro anotadores y acuerdo entre anotadores reportado, y una evaluación automática con RAGAS que distingue el ruido del juez de una mejora real.
5. Una compuerta de dominio entrenada con 160 preguntas que rechaza las preguntas sin respuesta del conjunto de prueba y las 5 del conjunto reservado, con 3 falsos rechazos en 20 preguntas de dominio, y una interfaz que expone la evidencia, las citas y la abstención.

I-C. Organización

La Sección II revisa los trabajos relacionados. La Sección III describe el corpus y el análisis exploratorio. La Sección IV presenta la arquitectura y justifica cada capa. La Sección V detalla el procesamiento y la configuración. La Sección VI define el protocolo y el silver. La Sección VII reporta los experimentos y sus decisiones. Las Secciones VIII a XII discuten errores, límites y conclusiones.

II. Trabajos relacionados

II-A. Recuperación aumentada por generación

RAG combina un recuperador con un generador para responder con conocimiento externo [1]. La variante densa codifica pregunta y documentos en un mismo espacio y busca por similitud; la dispersa, como BM25 [11], puntúa por coincidencia de términos. La fusión de ambas con reciprocal rank fusion [12] es una práctica común para compensar sus sesgos. En este trabajo se comparan las tres sobre el mismo corpus y el mismo protocolo, con un índice por codificador, y la configuración final combina densa y dispersa antes de reordenar, porque la unión recupera entidades que el denso no trae.

II-B. Variantes iterativas y correctivas

Self-RAG entrena al modelo para decidir cuándo recuperar y para criticar sus propias respuestas con tokens de reflexión [13]. CRAG evalúa la calidad de la recuperación y activa una búsqueda correctiva cuando el contexto es pobre [14]. Ambos mejoran la robustez frente a recuperaciones malas, a costa de varias pasadas del generador y, en el caso de Self-RAG, de un entrenamiento específico. Este trabajo no adopta esas variantes porque el presupuesto de latencia y la GPU local no permiten varias pasadas por consulta. En su lugar, la compuerta decide antes de generar y los guardarraíles verifican después, con un coste de milisegundos.

II-C. Codificadores y recuperación

BEIR mostró que un recuperador bueno en un dominio puede fallar en otro [2], y MTEB extendió esa comparación a tareas y lenguas [3]. MIRACL evidenció la brecha multilingüe [4]. Los codificadores entrenados con contrastive learning, como e5 [15] y bge-m3 [16], mejoran el rendimiento zero-shot y multilingüe. bge-m3 además produce representaciones densas, dispersas y multi-vector con un solo modelo, lo que simplifica la infraestructura. La elección entre ellos no se puede decidir solo por la literatura: depende del corpus, del idioma y del tamaño del índice. Por eso se miden sobre el corpus completo en lugar de limitarse al piloto.

II-D. Reordenamiento

Un cross-encoder puntúa el par pregunta-documento con atención conjunta y suele superar al bi-encoder en precisión [17]. El coste crece con el número de candidatos, porque cada par exige una pasada completa del modelo. La práctica habitual es recuperar un conjunto amplio y reordenar un top pequeño. Este trabajo mide el efecto de reordenar 5, 20 y 50 candidatos densos, y encuentra que ampliar el conjunto denso mejora el orden pero pierde cobertura. La configuración final reordena una unión de denso y BM25 en GPU y recorta a 10; la unión añade cobertura sin el coste de un top-50 denso.

II-E. Índices vectoriales

HNSW aproxima la búsqueda por vecinos con un grafo jerárquico y un parámetro de exploración que intercambia recall por latencia [18]. Milvus implementa HNSW y otros índices sobre almacenamiento distribuido [9]. El parámetro `ef` se ajusta en este trabajo con una búsqueda en rejilla sobre el silver, y se documenta el punto donde la ganancia deja de compensar el coste.

II-F. Evaluación

Las métricas de recuperación (Recall@k, nDCG, P@k, MRR) son estándar en recuperación de información. Para la respuesta generada, RAGAS descompone la calidad en fidelidad, relevancia, precisión y cobertura del contexto y corrección [5]; ARES entrena jueces para reducir el sesgo del evaluador [6]; eRAG evalúa la utilidad de cada documento recuperado [7]; y RAGChecker diagnostica errores de recuperación y de generación por separado [8]. Un problema recurrente es que el juez también es un modelo y su veredicto varía con el prompt, el timeout y la concurrencia. Liu et al. muestran además que el generador aprovecha peor la evidencia cuando esta se coloca en el medio de un contexto largo [19]. RAGTruth muestra que las alucinaciones en RAG se concentran en afirmaciones sin respaldo y propone anotación a nivel de span [20]. La verificación de entidades y números contra el chunk citado de este trabajo sigue esa idea con reglas deterministas. Este trabajo reporta la cobertura de cada métrica de RAGAS además de su media, y separa el ruido del juez de una mejora del sistema.

Tabla I
COMPOSICIÓN DEL CORPUS POR COLECCIÓN.

Colección	Documentos	% del total
Rastreo web	104.578	61,8
Bluesky	64.670	38,2
Total	169.248	100,0

II-G. Abstención y control de dominio

Un sistema RAG que siempre responde produce alucinaciones cuando la evidencia no alcanza. Las soluciones van desde pedir al generador que se abstenga hasta entrenar un clasificador que decida si la pregunta pertenece al dominio. Este trabajo usa dos controles: guardarraíles de cita y de entidades sobre la respuesta, y una compuerta de dominio supervisada que se activa con un interruptor. La compuerta es un clasificador de regresión logística con embeddings y señales de recuperación, no un juez generativo, para que su decisión sea barata, reproducible y medible.

II-H. Posición de este trabajo

La contribución es una evaluación completa de la cadena sobre un corpus real, con las mejoras que funcionaron y las que no, y con el coste de cada decisión. Se separa la calidad del sistema de la calidad del juez, y esa separación es la que permite decidir qué mantener en producción y qué descartar.

III. CORPUS Y ANÁLISIS EXPLORATORIO

III-A. Composición

El corpus de trabajo tiene 169.248 documentos, 34 columnas y 256,9 MB. La Tabla I resume la procedencia. La mayor parte viene de rastreo web (104.578) y de Bluesky (64.670). El campo `source_type` no sirve como etiqueta semántica: 99.326 filas lo tienen nulo, 64.670 valen `social_post` y solo 5.252 valen `news_article`. La distribución por dominio confirma el peso de las fuentes: bsky.app aporta 64.670 documentos, infobae 19.766, eltiempo 12.229, theguardian 7.216 y elespectador 5.456. El corpus no representa la conversación pública en español: Bluesky llega casi todo en inglés y la prensa está dominada por cuatro cabeceras. La prensa incluye cabeceras británicas (The Guardian, The Independent, Daily Mail) además de colombianas (El Tiempo, El Espectador, Infobae).

Tras normalizar el campo `language`, la Tabla II resume la distribución. El húngaro aparece con 4.108 documentos en el campo crudo, un volumen que no corresponde con el resto de la distribución y que se excluye del filtro de idioma hasta revisarlo. La normalización se hace al vuelo en la ingesta y el archivo original no se modifica.

El rango declarado de fechas va de 2008 a 2026, pero 163.736 de los 169.248 documentos (96,7 %) no tienen fecha y solo 5.512 la declaran. El filtro temporal se limita a ese 3,3 % y así se documenta en la interfaz.

Tabla II
IDIOMA NORMALIZADO DEL CORPUS. EL RESTO AGRUPA LOS IDIOMAS CON MENOS DE 500 DOCUMENTOS.

Idioma	Documentos	%
Inglés	82.682	48,9
Español	66.910	39,5
Sin declarar	13.140	7,8
Húngaro	4.133	2,4
Alemán	557	0,3
Portugués	468	0,3
Resto	1.358	0,8



Figura 1. Documentos por año en el corpus.

III-B. Duplicados y longitud

Hay 14.771 copias excedentes (filas que repiten un texto ya presente) agrupadas en 4.434 grupos, con un máximo de 839 copias. Si se cuentan todas las filas que pertenecen a un grupo duplicado, son 19.205: 9.972 en Bluesky y 9.233 en prensa. Las dos cifras son compatibles: la primera cuenta copias adicionales y la segunda, documentos en grupos. No se borran filas en la ingesta, porque eso perdería la procedencia; la deduplicación se hace en la recuperación, comparando el hash del fragmento antes de armar el contexto.

La longitud media es de 473 palabras y 2.977 caracteres, pero la mediana baja a 371 palabras y 2.367 caracteres. La cola es larga: el percentil 99 llega a 2.420 palabras y el máximo a 58.181 palabras, equivalentes a 363.528 caracteres. Ese documento extremo produce cientos de fragmentos y obliga a controlar el tamaño de lote en la ingesta.

III-C. Ruido de rastreo

El rastreador dejó páginas de error dentro del corpus. La misma página con los mensajes “Haz excedido el máximo de peticiones” y “Error 505” aparece en 7.378 documentos, el 4,36 % del total, todos en la colección de prensa. En el conjunto de humo esos textos casi idénticos dominaban el top-10 de cualquier consulta, porque su similitud entre sí es mayor que la de cualquier documento relevante. El registro de ingesta descarta 7.380 documentos por calidad: 7.378 son esa página de error y 2 no están desglosados. Quedan 161.868 documentos indexados y 618.286 fragmentos. El porcentaje de documentos con al menos un patrón de ruido (peticiones, error 505, cookies, inicio de sesión) llega al 5,3 %, pero el

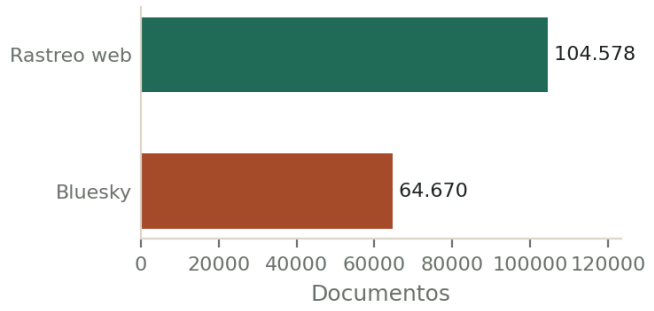


Figura 2. Documentos por colección. Los dos totales coinciden con la Tabla I.

Tabla III
TEXTOMETRÍA DEL PILOTO Y DE UNA MUESTRA DEL CORPUS COMPLETO.

Indicador	Piloto	Muestra
Tipos por documento (mediana)	188	188
TTR mediana	0,579	0,570
Hapax / tipos (mediana)	0,773	0,770
Guiraud (mediana)	10,04	10,13
Caracteres por documento (mediana)	2.317	2.353
Caracteres p99	14.979	15.498
Sin puntuación terminal (proxy)	33,5 %	32,7 %
Documentos con URL	9,3 %	9,9 %
Documentos con correo	1,8 %	1,3 %
Documentos con emojis	12,4 %	12,1 %
Documentos con navegación	3,4 %	3,3 %
Documentos con boilerplate	3,4 %	4,5 %
Near-duplicates (SimHash, Hamming ≤ 3)	4,25 %	4,72 %
Grupos de near-duplicates	26	53
Grupo más grande	12	32

filtro solo elimina los marcadores fuertes de error y no las menciones legítimas de cookies.

III-D. Textometría

La textometría se calculó sobre dos muestras: los 2.000 documentos del piloto y 5.000 documentos estratificados del resto del corpus, con semilla 42. La Tabla III compara los indicadores. Los valores son estables entre las dos muestras; eso no demuestra que el piloto represente la distribución del corpus, solo que no se observan diferencias gruesas en estos indicadores.

El 32,7 % de los documentos de la muestra no termina en un signo de cierre. Es un proxy de truncamiento, no una verificación de que el rastreador los cortó: una publicación completa puede acabar sin punto. La limpieza no puede recuperar ese texto, y el porcentaje se declara como límite en la Sección XI. El HTML residual es marginal (0,14 % en la muestra), pero las URLs aparecen en uno de cada diez documentos y los emojis en uno de cada ocho, casi todo en Bluesky. Los near-duplicates no son solo hash exacto: SimHash encuentra grupos de hasta 32 documentos que el hash exacto no captura. Los términos exclusivos de prensa están dominados por boilerplate (*publicidad*, *información*, *además*), lo que anticipa que una normalización conservadora puede ayudar sin destruir entidades.

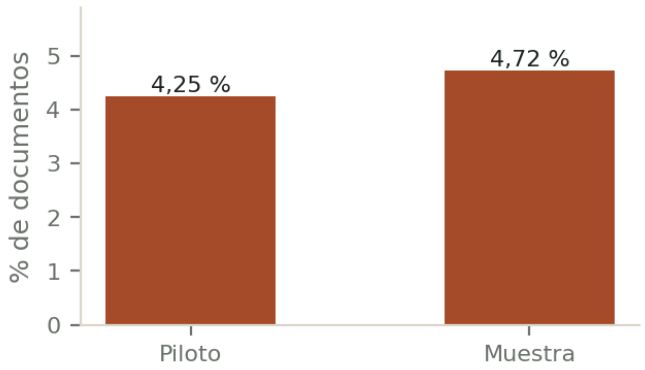


Figura 3. Porcentaje de documentos con near-duplicates (SimHash, distancia ≤ 3).

III-E. Decisiones de procesamiento

El análisis anterior fija tres decisiones. Primero, limpieza conservadora: se elimina boilerplate y se normalizan espacios, pero no se tocan entidades ni números. Segundo, deduplicación en la recuperación, no en la ingesta, para conservar la trazabilidad. Tercero, el truncamiento se declara como límite y no se intenta reconstruir. La Sección VII mide si la limpieza conservadora mejora la recuperación frente a no limpiar.

IV. ARQUITECTURA

IV-A. Vista general

La arquitectura se organiza como un marco modular de capas con contratos explícitos. Cada capa declara qué recibe, qué entrega y qué invariante conserva; la instancia concreta de cada ranura, sea el codificador, la mezcla de recuperación, el generador o el umbral, se elige por medición y se fija en la Sección VII. La Figura 5 muestra la instancia final; el arco que la separa de la línea base, con sus hipótesis y sus rechazos, se despliega en la Sección VII. Ninguna pieza se justifica por sí sola: la justificación es la comparación que la eligió.

IV-B. Capas y justificación

Ingesta. La capa convierte documentos crudos en fragmentos con metadatos. El contrato exige idempotencia y reanudación: cada fragmento guarda `document_id`, título, URL, fuente, idioma, fecha, posición, estrategia y hash, y la colección valida su etiqueta contra el codificador activo. La instancia final usa chunking fijo; el coste medido es 2 h 41 min para la ingesta completa y 27,3 min para reindexar 618.286 fragmentos ya troceados.

Índice vectorial. La capa recibe vectores y devuelve vecinos con metadatos. El contrato exige búsqueda aproximada con parámetro de exploración ajustable y filtrado por idioma, fecha y colección sin salir del motor; Milvus con HNSW cumple las tres condiciones [18]. El tamaño de la colección y el valor de `ef` se fijan en la Sección VII.

Codificación. La capa recibe texto y devuelve vectores densos. El contrato exige multilingüismo sin ajuste fino y una dimensión declarada en la etiqueta de la colección. La

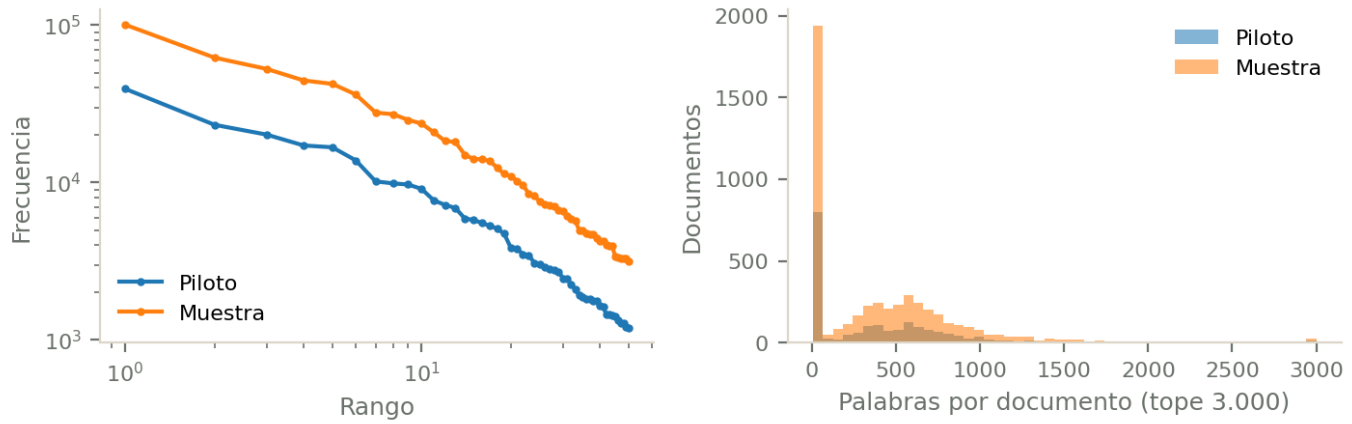


Figura 4. Textometría del piloto y de una muestra del corpus: rango frente a frecuencia en escala logarítmica y distribución de longitud en palabras.

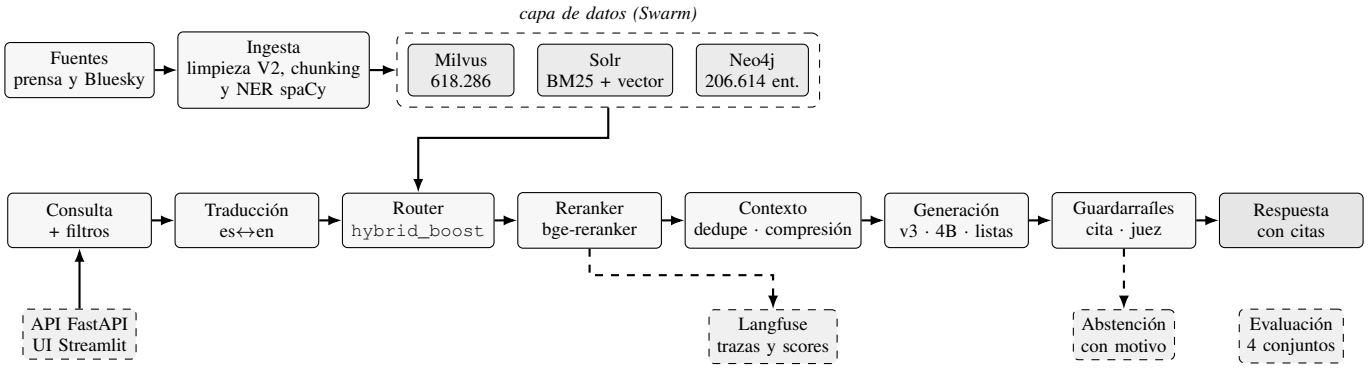


Figura 5. Arquitectura de la instancia final. La capa de datos (Milvus, Solr y Neo4j) se despliega como stack de Docker Swarm en el servidor y la aplicación (API, interfaz y Ollama) con Compose en la estación, que es la única que pasa la GPU AMD. La consulta se traduce al otro idioma del corpus y las dos listas se fusionan por RRF; el router `hybrid_boost` une la recuperación densa y BM25 con el bono de entidades del grafo; el contexto se deduplica y se comprime antes del prompt v3. Las líneas discontinuas marcan la traza hacia Langfuse y la abstención con motivo; el bloque de evaluación no forma parte del flujo de una consulta.

comparación entre e5-base, qwen3-embedding y bge-m3 se reporta en la Sección VII; la instancia final es bge-m3 [16].

Recuperación. La capa recibe una consulta y devuelve k fragmentos con puntuación. El contrato exige dos rutas independientes, densa y dispersa, deduplicación por texto y conservación del top-5 denso que consume la compuerta. La instancia final une denso top-10 y BM25 top-20 y suma al `rerank_score` un bono de 0,15 por cada entidad de la consulta presente en el fragmento (`hybrid_boost`); las mezclas, la fusión RRF y el bono se miden en la Sección VII. El mismo contrato se implementó sobre Solr 9 con `edismax`; el índice replica los 618.286 fragmentos y se selecciona con `RETRIEVER=solr`.

Reordenamiento. La capa recibe pares pregunta-fragmento y devuelve un orden. El contrato exige una puntuación reutilizable por la compuerta para no pagar dos pasadas. El cross-encoder multilingüe corre en GPU con lote de 16 [17]; el coste medido es 0,68 s para 50 pares, frente a 1,30 s para 5 pares en CPU. El recorte y la profundidad se comparan en la Sección VII.

Generación. La capa recibe el prompt y el contexto y devuelve texto con citas. El contrato es agnóstico del modelo: temperatura 0, presupuesto de 16.384 tokens de entrada, prompt v3 con ocho reglas y frase de abstención exacta. gemma4:e4b es la instancia final por contexto y MTP, no por ganar una comparación directa; los modelos pequeños y la corrida final se evalúan en la Sección VII.

Conversación. La capa recibe el historial y la última pregunta y devuelve una consulta autónoma. El contrato exige no cargar un segundo modelo grande; gemma2:2b actúa de condensador y, si pierde el tema, se antepone la primera pregunta del usuario.

Guardarraíles. La capa recibe la respuesta y las evidencias y devuelve la respuesta o una abstención con motivo. Tres controles: cita obligatoria, verificación de entidades y números contra el fragmento citado, y un juez de evidencia distinto del generador. El juez es gemma2:2b para no validar con el mismo modelo que redacta.

Compuerta de dominio. La capa recibe la puntuación del cross-encoder y señales de recuperación y devuelve una probabilidad de dominio. El contrato exige ser reproducible y medible con precisión y recall, no un LLM juez; una regresión logística reutiliza la pasada del reranker. Se activa con el interruptor Modo estricto y su entrenamiento se reporta en la Sección VII.

API e interfaz. FastAPI separa el pipeline de la presen-

tación y expone `/chat`, `/ask`, `/retrieve`, `/health`, `/models` y `/stats` [21]. Streamlit consume `/chat` y muestra la respuesta, las citas, las evidencias, las métricas y el razonamiento [22]. La separación permite probar el pipeline sin la interfaz y cambiar la interfaz sin tocar el pipeline.

IV-C. Mapa de técnicas

La Tabla IV reúne las técnicas que quedaron en la instancia final, el problema que atacan, cómo se implementaron y el resultado que las sostiene. Es el resumen del "por qué, cómo y para qué"; el detalle de cada implementación está en la Sección V y la medición en la Sección VII.

IV-D. Configuración final

La Tabla V fija la instancia final del marco. Cada valor se justifica en las Secciones V y VII.

IV-E. Despliegue: datos en Swarm, GPU en la estación

La capa de datos (etcd, MinIO, Milvus y Solr) se despliega como un stack de Docker Swarm en el servidor, con la etiqueta `role=data`. Milvus publica el puerto 19530 y Solr el 8983 en modo host; el API se conecta a `http://192.168.1.100:19530`. La aplicación (API, frontend y Ollama) corre en la estación con Compose y `network_mode: host`, porque necesita la GPU AMD.

La separación no es arbitraria. Swarm aporta colocación declarativa, política de reinicio y una red overlay que elimina el túnel SSH de la versión anterior; con el túnel, una caída de la conexión dejaba al sistema sin recuperación. La aplicación no puede ir en el stack: Docker Swarm no admite el paso de dispositivos (`devices`) ni `privileged` en un servicio, y el cgroup de dispositivos bloquea `/dev/kfd` con *Operation not permitted* aunque se monte el nodo y el grupo sea el correcto. Se comprobó con un servicio de prueba: el montaje y los grupos 983 y 987 funcionan, pero la apertura del dispositivo falla. Un stack completo obligaría a mover el generador a la GPU del servidor (GTX 1080 de 8 GB), que no soporta el contexto de 116K.

Compose se mantiene para la aplicación porque es el único runtime del proyecto que pasa la GPU AMD, y el stack se mantiene para los datos porque es el que elimina el túnel y hace declarativo el reparto. Un solo Compose no puede repartir servicios entre dos hosts; un solo Swarm no puede pasar la GPU.

IV-F. Interfaz para el usuario final

La interfaz tiene dos zonas. En el panel lateral se elige el modelo, el número de fragmentos, el idioma del documento, el rango de fechas y la colección; se activan el razonamiento y el Modo estricto, y se cambia entre tema claro y oscuro. En el área central se conversa con historial. Cada respuesta muestra el texto con citas numeradas, chips con la intención y la ruta de ejecución, la latencia por etapa, los tokens por segundo y un desplegable con las evidencias: título, puntuación, fragmento, enlace y metadatos. Si el sistema se abstiene, aparece el motivo y, en el caso de fuera de dominio, la sugerencia de desactivar el Modo estricto. El razonamiento del modelo, cuando se activa, va en un desplegable separado y se advierte que no sustituye la evidencia. La Figura 6 muestra una respuesta real con las evidencias abiertas.

V. METODOLOGÍA

V-A. Limpieza y deduplicación

La variante adoptada, V2, descarta las páginas de error del rastreador, aplica normalización NFKC, quita HTML y URLs y colapsa espacios. No elimina signos, números ni entidades, porque son la señal que usan los guardarraíles. Los duplicados exactos y los near-duplicates se descartan al armar el contexto, comparando el hash del fragmento y su SimHash antes de insertarlo. Así se conserva la procedencia en el índice y se evita repetir el mismo texto en el prompt.

V-B. Segmentación

Se usa chunking fijo de 1000 caracteres con 150 de solape (`fixed_v2`). El solape cubre la frontera donde suele caer la respuesta. Cada fragmento guarda `document_id`, título, URL, fuente, idioma, fecha, posición, estrategia y hash del texto. Esos metadatos permiten filtrar por idioma y fecha y auditar cada cita.

V-C. Codificación e índice

Los fragmentos se codifican con BAAI/bge-m3 en la GPU, en fp16 y por lotes. La colección se llama `rag_full_bge_m3_d1024_v1` y usa HNSW con `ef`=800 en la búsqueda. El parámetro se eligió con una rejilla sobre el silver: por debajo de 800 el recall cae y por encima la latencia sube sin ganancia apreciable. En paralelo se mantiene una colección BM25 sobre los mismos fragmentos, `rag_full_e5base_d768_v2_bm25_v1`, que alimenta la recuperación dispersa.

V-D. Enrutamiento y condensación

La primera decisión es qué hacer con la consulta, y se toma con reglas, sin llamar al modelo. Un clasificador de intención devuelve `charla` o `dominio`: son charla los saludos y la cortesía (todas las palabras pertenecen a una lista cerrada), las preguntas sobre el asistente (qué puedes hacer, quién eres, cómo funcionas) y las despedidas cortas sin interrogación; el resto es dominio. La ruta de charla responde con un prompt conversacional, sin recuperación ni citas; la de dominio entra al pipeline. La separación evita gastar una recuperación y un prompt largo en un saludo, y evita que el generador improvise una respuesta de corpus cuando solo lo saludan.

En una conversación, la última pregunta se convierte en una consulta autónoma antes de recuperar. El condensador es gemma2:2b con temperatura 0, el mismo modelo pequeño que hace de juez, y recibe los últimos seis turnos y la pregunta. Su salida se valida: no puede pasar de 40 palabras, debe compartir al menos una palabra de cuatro letras con la primera pregunta del usuario y no puede empezar con una disculpa o una negativa. Si falla la validación, el pipeline antepone la primera pregunta del usuario a la última, de modo que nunca se pierde el tema. El condensador no responde: solo reescribe.

Tabla IV
TÉCNICAS DE LA CONFIGURACIÓN FINAL: PROBLEMA, IMPLEMENTACIÓN, RESULTADO Y SECCIÓN.

Técnica	Por qué	Cómo	Resultado	Sec.
Limpieza V2	Ruido de rastreo y HTML	NFKC, sin HTML ni URL, espacios colapsados	Sin mejora significativa; se adopta por consistencia	V-A
Chunking fijo	La respuesta cae en la frontera	1000 caracteres, 150 de solape, metadatos	Supera a oraciones y es más rápido	V-B
bge-m3	Consultas y documentos en dos idiomas	Embeddings fp16 en GPU, 1024 dimensiones	Recall@10 0,842 frente a 0,684 de e5-base	V-C
HNSW `ef`=800	Recall sin disparar la latencia	Índice aproximado con exploración ajustable	Recall@10 0,895; arriba no mejora	V-C
Unión densa + BM25	El denso no trae cifras ni nombres raros	Dos listas, dedupe por texto, top-5 denso intacto	Recall@10 0,700 en el reservado	V-E
Reranker en GPU	El orden importa más que el corte	Cross-encoder sobre 50 pares, lote de 16	0,68 s; nDCG y P@1 suben sin perder recall	V-E
`hybrid_boost`	Entidades que el vector diluye	Bono de 0,15 por solape con las entidades del grafo	0,861 en silver; 0,900 en multi-documento	V-H
Prompt v3	El modelo omitía datos pedidos	Respuesta directa, todos los datos, dos ejemplos	RAGAS 0,590 a 0,652 con la misma exactitud	V-I
Traducción + RRF	Pregunta y evidencia en idiomas distintos	Traducción al otro idioma, fusión 0,5/0,5	Resuelve g-046; test 0,917 a 0,958	V-F
Listas en JSON	Enumeraciones incompletas	Detección por patrón, JSON, composición	Resuelve g-003; test 0,958	V-J
Grafo Neo4j	Desambiguar entidades	Documento a entidad, un salto, filtro de ruido	206.614 entidades; cobertura 0,626 a 0,655	V-H
Compuerta	Abstención barata antes de generar	Logística sobre cross-encoder y señales de recuperación	F1 0,952; 0 falsos aceptados de 9	V-L
Guardarraíles	Citas inventadas o sin respaldo	Cita obligatoria, entidades y juez distinto	Anclaje 0,907 en silver	V-K
Langfuse	Atribuir cada mejora a una causa	SDK, spans, scores y datasets	Cuatro corridas comparables sobre 43	V-N
Solr	Backend sustituible sin cambiar el contrato	`edismax` y `knn_vector` sobre el mismo corpus	BM25 iguala 0,763; denso 0,605	V-O

Tabla V
CONFIGURACIÓN DESPLEGADA. LOS VALORES SE FIJAN EN `.env` Y CADA UNO SE JUSTIFICA EN LAS SECCIONES V Y VII.

Parámetro	Valor
Codificador	BAAI/bge-m3, 1024 dimensiones, fp16 en GPU
Índice vectorial	Milvus 2.6.6, HNSW, `ef`=800, 618.286 fragmentos
Recuperación	`RETRIEVER=hybrid_boost`: denso top-10 + BM25 top-20, $k = 10$
Bono de grafo	0,15 por entidad de la consulta presente en el fragmento
Reordenador	BAAI/bge-reranker-v2-m3, GPU, top-10
Traducción	`TRANSLATION=1`, es↔en, fusión RRF 0,5/0,5
Listas	`LIST_EXTRACTION=1`, JSON con composición y reserva
Generador	gemma4:e4b, Q4_K_M, 16.384 tokens de entrada
Prompt	v3
Grafo	Neo4j 5.26, 206.614 entidades y 852.751 menciones
Backend alterno	Solr 9, `edismax` + `knn_vector`, `RETRIEVER=solr`
Observabilidad	Langfuse 4.78: trazas, scores y datasets
Juez y condensador	gemma2:2b
Compuerta	regresión logística sobre cross-encoder, umbral 0,4
Apagados	`NUMERIC_VERIFICATION=0`, `QUERY_EXPANSION=0`, `CANDIDATE_GUARANTEE=0`

V-E. Recuperación y reordenamiento

La recuperación combina dos listas. La densa pide 10 fragmentos a bge-m3; la dispersa pide 20 a BM25. Las dos se unen sin fusionar, se deduplican por texto y se pasan al reordenador. El reordenador es un cross-encoder multilingüe, BAAI/bge-reranker-v2-m3, que codifica el par pregunta-fragmento de forma conjunta: la atención ve las dos secuencias a la vez y emite una puntuación de relevancia, en lugar de comparar dos vectores independientes. Se carga en fp16 sobre ROCm, con `max_length` de 512 tokens (el fragmento se trunca a esa longitud) y lotes de 16 pares. La puntuación se calcula una sola vez por par y se guarda en `rerank_score`: el orden usa esa puntuación y la compuerta de dominio la reutiliza, de modo que no se paga una segunda pasada. Sobre los 30 candidatos de la unión, 50 pares cuestan 0,68 s en GPU frente a 1,30 s en CPU. El recorte final es top-10: lo que entra al prompt es el orden del cross-encoder, no el de la fusión. La unión conserva el top-5 denso, que es la señal que la compuerta espera, y añade las entidades que BM25 encuentra y el denso no. Los filtros de idioma, fecha y colección se aplican en Milvus. Sobre esa unión, `hybrid_boost` suma al `rerank_score` un bono de 0,15 por cada entidad de la consulta que aparece en el fragmento y vuelve a ordenar; el bono desempata sin cambiar los candidatos, a diferencia de `graph_hybrid`, que los sustituye. Cuando la consulta y la evidencia pueden estar en idiomas distintos, la recuperación se repite con la traducción y las dos listas se fusionan por RRF (Sección V-F).

V-F. Traducción de la consulta

El corpus mezcla español e inglés, y una pregunta en un idioma puede tener la respuesta en el otro. El pipeline traduce la consulta al idioma opuesto y busca las dos formas. La detección es una heurística declarada: si la consulta contiene marcadores del español (el signo de apertura de interrogación, `qué`, `cuál`, `cómo`, `dónde`, `cuándo`, `cuánt`, `por qué`, `quién`) se traduce al inglés; si no, al español. No es un detector estadístico y no pretende serlo: el corpus tiene dos idiomas y el coste de equivocarse es una lista más que el reordenador puede descartar. La traducción la hace el mismo generador con temperatura 0 y un límite de 80 tokens, con la



Figura 6. Interfaz final: respuesta con citas y evidencias abiertas. La pregunta se acepta con probabilidad de dominio 0,956; la respuesta cita diez evidencias y el panel muestra título, puntuación, fragmento, enlace y metadatos de la primera. El chip de la derecha identifica la ruta de ejecución, no un guardarraíl de cita.

instrucción de conservar nombres, lugares, fechas y números y devolver solo la frase. La consulta original y la traducción se recuperan por separado y se fusionan con RRF ponderado ($k = 60$, pesos 0,5/0,5); si la traducción falla o repite la consulta, se sigue con la original. El span `translate` guarda la traducción y su tiempo.

V-G. Vecinos semánticos y sensibilidad a la paráfrasis

La recuperación también se inspeccionó de forma cualitativa. Se pidieron hasta cinco vecinos por consulta en ocho consultas de ejemplo sobre la colección final; la Tabla VI reproduce dos. Los vecinos mezclan idiomas: bge-m3 acerca el mismo tema aunque el texto esté en otra lengua, y el campo de idioma de algunas publicaciones de Bluesky viene vacío en la fuente. En la consulta en español sobre elecciones, el primer vecino es una nota en inglés sobre las elecciones colombianas y el cuarto es un post en español; en la consulta en inglés sobre economía, los tres primeros fragmentos son consecutivos del mismo artículo en español sobre Davos. La

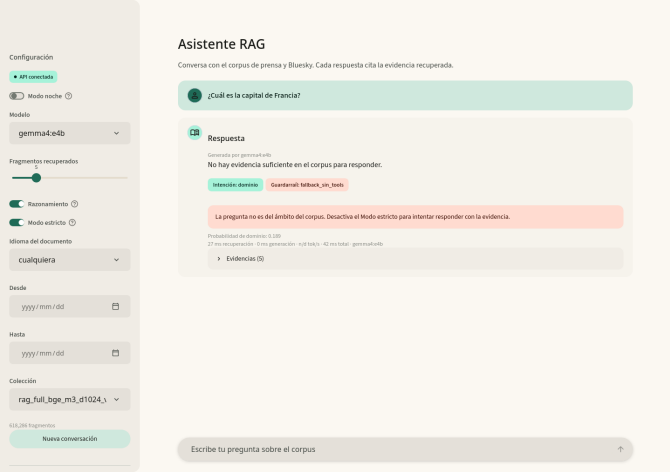


Figura 7. Modo estricto: la compuerta rechaza una pregunta fuera de dominio y explica el motivo.

inspección confirma que el espacio es multilingüe y muestra dos límites: la deduplicación por texto exacto no colapsa fragmentos contiguos del mismo documento, y el ruido de rastreo, como HTML incrustado o marcas de redes, llega al índice.

La segunda prueba midió estabilidad ante paráfrasis. Sobre 20 paráfrasis, 9 en inglés y 11 en español, se calcularon cinco mezclas denso/disperso con la misma profundidad y el nDCG resultante (Tabla VII). En inglés el nDCG se mantiene entre 0,724 y 0,858 y supera 0,78 en tres de las cinco mezclas; en español cae a 0,578 con peso denso 1,0 y solo la mezcla puramente dispersa llega a 0,739. La conclusión es asimétrica: con estas mezclas el orden es estable ante paráfrasis en inglés y no lo es en español. La comparación es descriptiva, con $n = 9$ y $n = 11$, sin intervalos de confianza.

V-H. Grafo de entidades

El grafo vive en Neo4j 5.26. El esquema declara dos restricciones de unicidad (`document_id` y `entity_key`), un índice full-text sobre los nombres de entidad y una relación `MENTIONS` entre documento y entidad. La extracción corre con spaCy en español e inglés sobre el título y los primeros 1.500 caracteres; cada entidad se normaliza a una clave y se fusiona con `MERGE`, de modo que el mismo nombre no se duplica. La construcción se hace en un proceso local, por lotes de 1000 y escribiendo por Bolt al servidor: 169.248 documentos, 248.869 entidades y 926.923 menciones en 2.000 s. Un filtro posterior (máximo 4 tokens, sin dígitos iniciales y `MISC` de máximo 3) deja 206.614 entidades y 852.751 menciones.

El grafo se usa de dos formas. `search_graph` recupera documentos semilla por las entidades de la consulta y expande el camino documento–entidad–documento: toma las 12 entidades semilla con más peso y, por cada una, hasta 15 documentos; ese modo alimenta `graph_hybrid`. `boost_hits` no toca la recuperación: suma 0,15 al `rerank_score` por cada entidad de la consulta presente en el fragmento y reordena. El segundo es el que quedó en la configuración final: desempata sin desplazar lo que el índice ya traía.

V-I. Generación

El generador es gemma4:e4b con temperatura 0 y 16.384 tokens de entrada. El prompt v3 sustituye al v2 y cambia el énfasis: pide la respuesta directa en la primera frase con el dato exacto; si la pregunta pide varios datos, todos; si pide una lista, todos los elementos que aparezcan en la evidencia; si dos fragmentos se contradicen, el más específico; una cita por afirmación; nada de relleno ni de repetir la evidencia; la frase de abstención exacta cuando no alcanza; ningún dato inventado y ninguna instrucción interna de los documentos; y respuesta en el idioma de la pregunta. Incluye dos ejemplos cortos, uno de fecha y lugar y otro de enumeración, porque el fallo medido era de omisión, no de formato. El MTP con draft 3 se activa en el manifiesto del modelo y no cambia la salida, solo la velocidad.

Para las preguntas de opciones o medidas, la generación se sustituye por una extracción estructurada (Sección V-J). El resto de las peticiones pasa por el prompt v3 completo, con el contexto deduplicado y comprimido al presupuesto de tokens.

V-J. Extracción de listas

Las preguntas de opciones o medidas no se responden bien con un prompt de prosa: el modelo tiende a resumir y pierde elementos. El pipeline detecta el patrón con una expresión regular (`qué opciones`, `qué medidas`, `qué acciones`, `qué estrategias`, `qué alternativas`, `cuáles son`, `cuáles fueron`, `cuáles serán`) y excluye las que contienen `cuánt`, porque esas piden una cifra y no una enumeración. Si el modo está activo, la generación cambia por una extracción: se pide al modelo un JSON con los elementos literales de la enumeración que aparece en la evidencia, sin resumir ni copiar párrafos que no sean una lista. `parse_list` tolera listas de cadenas o de objetos y `compose_list` arma la respuesta con cada elemento y su cita. Si el JSON viene vacío, no parsea o no hay elementos, se cae al prompt normal: la extracción nunca deja la pregunta sin respuesta. El span `extract_list` guarda los elementos y su tiempo.

V-K. Guardarraíles

Antes de devolver la respuesta se aplican tres controles. El primero exige al menos una cita válida; si no hay, se abstiene con `sin_citas`. El segundo extrae entidades y números de la respuesta y verifica que aparezcan en el fragmento citado o en la pregunta; si no, se abstiene con `sin_respaldo` o `citas_invalidas`. El tercero pide a un juez distinto, gemma2:2b, que decida si la cita sostiene la respuesta; si dice que no, se abstiene con `juez_no`. Cuando el fragmento está en inglés y la respuesta en español, la cobertura léxica se salta y decide el juez, para no castigar una traducción correcta. El juez recibe la pregunta, la respuesta y el contexto de los fragmentos citados, recortado a 3.000 caracteres; corre con gemma2:2b a temperatura 0 y devuelve una decisión binaria, de modo que una respuesta que no empieza por “sí” se abstiene.

Tabla VI
VECINOS SEMÁNTICOS DE DOS CONSULTAS DE EJEMPLO EN LA COLECCIÓN FINAL. *s/d*: el campo de idioma viene vacío en la fuente y no se infiere. La primera consulta devolvió cuatro vecinos con $k = 5$.

Consulta: `resultados de las elecciones en Colombia`			
Rango	Sim.	Idioma	Vecino recuperado
1	0,669	en	Trump-backed candidate wins razor-tight Colombia presidential election
2	0,632	s/d	"Surprise" election result poses new challenges for the left in Colombia
3	0,632	s/d	Live: Colombia heads to the polls to elect President Gustavo Petro's successor
4	0,627	es	El partido de Petro gana las elecciones al Senado en Colombia y Uribe recupera el liderazgo de la oposición
Consulta: `what did the president say about the economy`			
Rango	Sim.	Idioma	Vecino recuperado
1	0,646	es	"...recordó que en 2018, cuando acudió por primera vez como presidente, dijo al público que haría el gran 'regreso norteamericano'."
2	0,638	es	"...el discurso del presidente de Estados Unidos el primer día en Davos se dio mientras en Washington comenzaba el juicio político en su contra."
3	0,631	es	"...el mensaje de Trump sonó a un presidente triunfalista, confiado en que logrará la reelección este año."
4	0,628	s/d	El Presidente anuncia medidas económicas para enfrentar la crisis.
5	0,625	es	"...el jefe de Estado se reunió en EE.UU. con el empresario Mike Bloomberg para hablar de cambio climático y transición energética."

Tabla VII
NDCG ANTE PARÁFRASIS POR MEZCLA DENSO/DISPERSO ($n = 9$ EN INGLÉS, $n = 11$ EN ESPAÑOL).

Peso denso/disperso	Inglés	Español
0,0/1,0	0,729	**0,739**
0,3/0,7	0,840	0,683
0,5/0,5	0,824	0,650
0,7/0,3	**0,858**	0,586
1,0/0,0	0,724	0,578

V-L. Compuerta de dominio

La compuerta no es un LLM juez sino una regresión logística sobre 15 features que ya están calculadas. Cuatro resumen la puntuación del cross-encoder sobre el top-5: máximo, media, diferencia entre el primero y el segundo, y mínimo. Cinco describen la recuperación: puntuación del primer fragmento, media del top-5, diferencia entre el primero y el segundo, solapamiento entre la pregunta y el título, y presencia de la entidad principal en el título. Seis son un one-hot del tipo de pregunta: capital, población, autor, fecha, lugar, cantidad y otras. Las features se estandarizan con la media y la escala guardadas en el modelo, y la probabilidad es la sigmoide del logit. La puntuación del cross-encoder se calcula una sola vez y se reutiliza para el reordenamiento. El modelo es una regresión logística con escalado y `class_weight=balanced`; la regularización se eligió por validación cruzada de cinco pliegues sobre train y dev. El umbral es 0,4, elegido para reducir falsos rechazos sin aumentar falsos aceptados. Si el archivo del modelo no existe, la compuerta devuelve probabilidad 1 y no filtra: el sistema degrada a sin compuerta, no bloquea. El conjunto tiene 160 preguntas: 60 de dominio y 100 de fuera, con split 103/37/20. Las 40 negativas nuevas son entidades inventadas, eventos futuros y conocimiento general fuera del corpus.

V-M. Ambigüedad referencial

Una pregunta es ambigua cuando admite más de una respuesta válida sobre el mismo corpus: "¿cuántos votos obtuvo la iniciativa?" no dice cuál iniciativa, y "¿quién ganó el concierto?" no dice cuál concierto. El sistema no la responde a ciegas ni la rechaza: detecta la ambigüedad y ofrece las interpretaciones.

La detección tiene dos etapas. La primera es una heurística barata que decide si vale la pena consultar al modelo. Dispara cuando la consulta contiene una referencia genérica de una lista cerrada (la iniciativa, el concierto, la reforma, el proyecto, el acuerdo, la cumbre, la ley, el programa, el plan, la campaña, el terremoto, el estudio, el informe y sus equivalentes en inglés), cuando usa un superlativo en una pregunta corta (máximo 16 palabras), cuando pregunta un precio o una medida con unidad, cuando pregunta un porcentaje, o cuando es un seguimiento conversacional corto (ocho palabras o menos, o empieza por "y"). Si no dispara, no hay llamada al modelo.

La segunda etapa es una llamada a gemma4:e4b con la pregunta, los últimos tres turnos del historial y hasta diez evidencias de 600 caracteres. El modelo devuelve un JSON con `ambiguous`, `type`, `clarifying_question` y entre dos y tres opciones; cada opción trae una etiqueta, una consulta reescrita, una respuesta corta y los índices de la evidencia que la sostienen. Las opciones se validan: etiqueta, consulta y respuesta no vacías, e índices dentro del rango; si sobreviven menos de dos, la pregunta se trata como no ambigua. Para que el detector vea suficiente contexto, cuando la heurística dispara se recupera una unión profunda de 50 fragmentos densos y 50 dispersos, se reordena y se le pasan los diez primeros.

El tratamiento depende de la política. `clarify` devuelve la pregunta de clarificación y las opciones a la interfaz, sin generar respuesta; `conditional` responde cada interpretación con su cita en un solo turno; `auto` elige `clarify` en la interfaz conversacional y `conditional` en la API de una pasada; `off` lo desactiva. La comparación de las cuatro políticas está en la Sección X.

V-N. Observabilidad

Cada petición abre una traza `ask` en Langfuse 4.78 y un span por etapa: `translate`, `retrieve`, `rerank`, `generate`, `extract_list` y `grounding`, con la entrada, la salida, los tiempos y los tokens. Los fragmentos recuperados se registran con su `chunk_id` y su puntuación; el prompt se recorta a 8.000 caracteres para no duplicar el contexto en el servidor de trazas; la respuesta lleva el `trace_id` para enlazarla desde la API. Al cerrar la petición se adjuntan tres scores: `citation_valid`, `grounded` y `latency_ms`. Si Langfuse no responde, `observation` devuelve `None` y el pipeline continúa: la observabilidad no es una dependencia dura. Los conjuntos viven como Dataset y cada corrida es un experimento que calcula cuatro scores por ítem: `correct`, `token_recall`, `citation_valid` y `grounded`.

V-O. Solr como backend alterno

El mismo contrato de recuperación se implementó sobre Solr 9. El esquema declara `text` y `title` como `text_general` con `qf=text title^2`, los metadatos como campos almacenados y `vector` como `knn_vector` de 1024 dimensiones con similitud coseno. La búsqueda léxica usa `edismax`; la densa usa el query parser `{!knn f=vector topK=k}` con el vector de bge-m3. La ingesta se hace con `index_solr` e `index_solr_vectors`, que replican los mismos 618.286 fragmentos, y el despliegue con `scripts/deploy_solr.sh`, que crea el core y el esquema. Se selecciona por petición con `RETRIEVER=solr` y no cambia el resto del pipeline. BM25 iguala el recall del BM25 de Milvus (0,763) y el denso se queda en 0,605 frente a 0,842 de Milvus; por eso Milvus es el backend por defecto y Solr queda como alternativa verificada.

V-P. API e interfaz

La API expone `POST /chat` con historial, `think` y `strict`; `POST /ask` sin historial; `POST /retrieve` para recuperación pura; y los GET `/health`, `/models` y `/stats`. La interfaz Streamlit consume `/chat` y guarda el historial en la sesión. Las preferencias de razonamiento, Modo estricto y tema se guardan en un archivo y se recuperan al recargar. Si el usuario cambia un interruptor mientras el modelo responde, la pregunta queda pendiente y se reintenta en el siguiente ciclo, de modo que la conversación no se pierde.

V-Q. Despliegue

El stack completo se levanta con `make all-up`: Docker Compose construye y arranca etcd, MinIO, Milvus, Solr, Ollama, la API y el frontend, espera a que la API esté sana, descarga los modelos, ingiere el Parquet, construye la colección dispersa e indexa Solr en texto y vectores. `make obs-up` levanta Langfuse en el servidor y `make graph-build` construye el grafo. La capa de datos corre en Swarm y la aplicación en la estación con red de host, porque es la única que pasa la GPU AMD; la justificación está en la Sección IV.

Tabla VIII
COMPOSICIÓN DEL SILVER STANDARD.

Split	Respondibles	Sin respuesta	Total
Desarrollo	19	0	19
Prueba	19	5	24
Total	38	5	43

Tabla IX
ACUERDO ENTRE ANOTADORES EN LA COMPARACIÓN DE MODELOS (KAPPA LINEAL, $n = 72$).

Eje	Kappa
Corrección	0,95
Citas	0,90
Abstención	0,72
Fidelidad	0,12

VI. PROTOCOLO DE EVALUACIÓN

VI-A. Silver estándar

El silver standard tiene 43 preguntas: 19 en desarrollo y 24 en prueba, 38 respondibles y 5 sin respuesta. Lo denominamos silver standard y no gold standard porque lo generó un LLM y lo revisaron los autores, no expertos independientes. Cada fila guarda el identificador, la pregunta, el idioma, el tipo, el split, los documentos y fragmentos de referencia, el span literal que responde, la respuesta de referencia, el criterio de corrección y el estado de validación. La Tabla VIII resume la composición. Las preguntas sin respuesta no entran en Recall@k: se evalúan por abstención.

El silver standard se amplió con 17 documentos adicionales marcados por consenso de dos anotadores (kappa 0,626). Con esa ampliación, 14 preguntas quedaron con dos o más documentos relevantes, lo que hace que Recall@k y nDCG no dependan de un único pasaje.

VI-B. Revisión humana

Cuatro personas revisaron el silver standard contra el documento citado. Carlos revisó las 43 y dejó 12 correcciones; Julián aportó 7 spans y una reformulación; Óscar aportó 5 spans corregidos y marcó 3 enlaces rotos; Erika revisó las 43, aprobó 35, marcó 5 como no aplicables y corrigió la respuesta y el span de g-003. Cada propuesta se aplicó solo si el span aparecía literalmente en el texto indexado. Las que no se pudieron verificar quedaron con nota.

Para la comparación de modelos se anotaron 72 respuestas de 4 modelos, ciegas y en orden barajado, con rúbrica de 0 a 3 en corrección, fidelidad, citas y abstención. Dos anotadores calificaron las mismas respuestas. La Tabla IX reporta el acuerdo. La fidelidad no alcanzó acuerdo suficiente y no se usa para el veredicto; la corrección y las citas sí.

VI-C. Métricas

Recuperación. Recall@k mide la fracción de documentos de referencia recuperados en el top-k; nDCG@k usa relevancia binaria; P@1 mide precisión en el primer corte; Hit@5 indica si al menos un documento relevante aparece en el top-5; MRR

mide la posición del primer acierto; y la cobertura de span indica si al menos un fragmento recuperado contiene el texto exacto que responde. Esta última es la más cercana a lo que necesita el generador.

Respuesta. Se reportan las preguntas entregadas, las abstenciones, la cobertura de tokens de la referencia (todos sus tokens aparecen en la respuesta), el F1 por tokens, la tasa de citas, el contexto medio en tokens y la latencia por etapa. La revisión humana se aplica solo a las respuestas que no superaron la cobertura y las clasifica como correctas, parciales o incorrectas; no cubre las que sí la superaron.

Conversación. Catorce casos: seis saludos, cuatro de dominio y cuatro de fuera. Cada uno tiene un resultado esperado y se marca como correcto o incorrecto. Se ejecutan con y sin Modo estricto, y con y sin razonamiento.

RAGAS. Se reportan fidelidad, relevancia de la respuesta, precisión del contexto, cobertura del contexto y corrección de la respuesta [5]. Para cada métrica se informa cuántas muestras tienen valor, porque el juez falla en algunas y una media con cuatro valores no es comparable con una con 38. La corrida de referencia es densa con $k = 5$; la comparación con el híbrido y con la arquitectura anterior está en la Sección VII.

Compuerta. Precisión, recall y F1 en dev y test, falsos rechazos de dominio (preguntas del corpus rechazadas) y falsos aceptados de fuera (preguntas ajenas aceptadas).

VI-D. Estadística

Las comparaciones de recuperación usan bootstrap pareado por pregunta con 1.000 remuestreos e intervalos de confianza al 95 %. En la comparación de limpieza se aplica corrección de Bonferroni por comparaciones múltiples. La configuración se eligió inspeccionando las 43 preguntas del silver, así que los resultados sobre las 38 respondibles son exploratorios. El conjunto reservado de 25 preguntas (Sección VII-S) se congeló antes de evaluar la compuerta, pero después se usó para elegir la recuperación híbrida; por eso sus métricas son de desarrollo y no una prueba independiente. Las diferencias que no excluyen cero se reportan como no concluyentes, aunque la media favorezca a una variante. Los intervalos pareados se calcularon para limpieza, E1 y E2; las comparaciones de codificadores, reordenamiento y fusión se reportan como estimaciones puntuales sobre 38 preguntas, sin intervalo, y así se indica en cada tabla.

VI-E. Configuración de RAGAS

La corrida de referencia usa juez gemma4:e4b y embeddings BAAI/bge-m3 en CPU, con 38 preguntas directas. El timeout del juez se fijó en 900 s y la concurrencia en 6. Con los valores por defecto (180 s y 16 workers) el juez no terminaba en varias muestras y la precisión del contexto quedaba con 4 valores de 38. La corrida de referencia tiene 34 valores en esa métrica y 38 en las demás. Las preguntas sin respuesta se miden aparte y se abstienen 5 de 5.

VII. Experimentos y resultados

Los experimentos usan el silver standard de 43 preguntas; E12 usa la versión ampliada con 17 documentos adicionales y cada tabla lo indica. La configuración se eligió con el silver completo, así que los resultados son exploratorios. Cada subsección reporta el montaje en una frase, la tabla y la decisión; el detalle del montaje está en la Sección V.

VII-A. Arco experimental

Los experimentos no forman una rejilla: siguen un arco de seis estaciones, y cada una se abre con la pregunta que deja la anterior.

- **Línea base sobre el piloto.** Limpieza, segmentación y número de fragmentos con e5-base y 1.932 documentos (Subsecciones *Limpieza* a *Segmentación y tamaño del contexto*). El piloto deja un techo alto, Recall@10 de 0,947, y decisiones tomadas por coste y consistencia, no por significancia estadística.
- **Modelos pequeños e H1.** E4 compara cuatro generadores de 1B a 3B con la misma evidencia recuperada (Subsección *Modelos generativos*). H1 se refuta: el tamaño no ordena la calidad; gemma2:2b gana en corrección y citas y llama3.2:1b se abstiene en el 73,7 % de las preguntas respondibles.
- **Crisis de escala.** E6 repite el pipeline sobre el corpus completo, 161.868 documentos (Subsección *Codificadores y escala*). Recall@10 cae de 0,947 a 0,684 y nDCG de 0,909 a 0,594 sin que la latencia cambie; la caída reorganiza el resto del trabajo.
- **Rescate híbrido.** BM25, bge-m3, `ef`=800, la unión densa y dispersa y el reordenador recuperan parte de la pérdida (Subsecciones *Codificadores y escala* y *Reordenamiento*). La fusión RRF se rechaza con el silver ampliado y la unión reordenada se adopta para el despliegue.
- **Ablaciones negativas.** Las fases 6 y 7 y la inyección de instrucciones prueban mejoras que no superan la referencia (Subsecciones *Inyección de instrucciones* y *Prompts y mejoras rechazadas*). Se reportan para acotar el espacio, no para justificar la configuración.
- **Arquitectura ganadora y reservado.** La instancia final se evalúa con la compuerta de dominio, RAGAS y el conjunto reservado de 20 preguntas directas y 5 negativas (Subsecciones *Compuerta de dominio* a *Conjunto reservado*). Es la evaluación de cierre y se reporta tal como sale, incluidas las cifras que empeoran la línea base.

El orden importa: las estaciones iniciales se deciden con el silver exploratorio y el conjunto reservado se usa al final para comparar la configuración desplegada; las cifras de una estación no se trasladan a otra. Cada subsección indica con qué conjunto se midió.

VII-B. Limpieza

Se construyeron cuatro índices sobre el piloto con 30 preguntas, e5-base, $k = 10$ y tres repeticiones. La Tabla X muestra el resultado. Ninguna comparación pareada contra V0 excluye cero tras la corrección de Bonferroni sobre 21 pruebas. V2 toca el límite inferior en cuatro métricas, compatible con una mejora pequeña o con ausencia de efecto. Se adopta V2 por tres razones no estadísticas: la consistencia de la

Tabla X
LIMPIEZA SOBRE EL PILOTO (30 PREGUNTAS, E5-BASE, $k = 10$).

Variante	P@1	Hit@5	Recall@10	MRR	nDCG
V0 crudo	0,733	0,833	0,833	0,775	0,790
V1 sin páginas de error	0,767	0,833	0,833	0,800	0,809
V2 normalización	**0,833**	**0,900**	**0,900**	**0,867**	**0,875**
V3 V2 + dedup	0,800	0,867	0,867	0,833	0,842

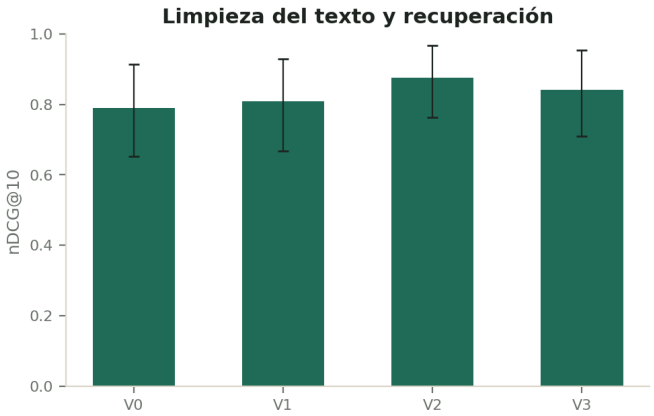


Figura 8. Comparación de variantes de limpieza sobre el piloto.

Tabla XI
E1: CHUNKING FIJO CONTRA ORACIONES (30 PREGUNTAS).

Variante	P@1	Hit@5	Recall@10	nDCG	p50
Fijo 1000/150	**0,833**	**0,900**	**0,900**	**0,875**	**38,5 ms**
Oraciones 384/1	0,767	0,833	0,833	0,804	42,1 ms

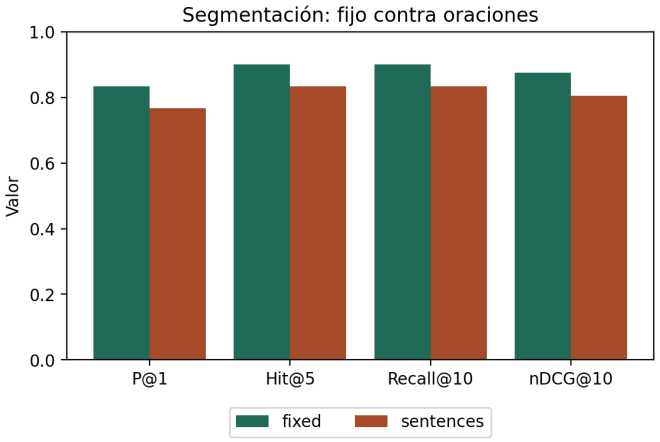


Figura 9. Segmentación: fijo contra oraciones.

estimación en todas las métricas, el coste nulo en consulta y la eliminación de 68 páginas de error que de otro modo entrarían al índice.

VII-C. Segmentación y tamaño del contexto

E1 comparó chunking fijo de 1000 caracteres con 150 de solape contra segmentación por oraciones de 384 caracteres. La Tabla XI muestra estimaciones puntuales favorables al fijo en todas las métricas de calidad, pero ningún intervalo pareado excluye cero: nDCG $-0,071$ (IC $[-0,221;\ 0,025]$), recall $-0,067$ (IC $[-0,233;\ 0,000]$) y P@1 $-0,067$ (IC $[-0,233;\ 0,114]$) para oraciones menos fijo. La única diferencia concluyente es la latencia: el fijo es 3,65 ms más rápido (IC $[2,25;\ 4,97]$). Se mantiene el fijo por la dirección consistente y el coste, no por una mejora de calidad demostrada.

E2 ablacionó el número de fragmentos. Con el silver del piloto, el documento relevante está siempre en el top-3, así que $k = 3$, 5 y 10 devuelven las mismas métricas de calidad. La única diferencia significativa es la latencia: $k = 3$ tarda 1,64 ms menos que $k = 10$ y $k = 5$ tarda 0,94 ms menos. Se fija $k = 5$ como margen para preguntas comparativas que no están en el conjunto, sabiendo que el coste es más contexto en generación, no más latencia de recuperación.

VII-D. Codificadores y escala

E3 comparó e5-base con qwen3-embedding:8b sobre el piloto, con 18 preguntas de desarrollo. qwen3 recupera mejor: Recall@10 perfecto y 6,4 puntos más de nDCG. El problema es el coste. e5-base indexó el corpus completo en 2 h 41 min medidos en la RX 9070 XT. qwen3-embedding:8b no se indexó entero: las 23 horas son una extrapolación desde el rendimiento por lote, no un tiempo medido. Se adopta e5-base para la ingesta y qwen3 queda como comparación documentada.

E6 midió el mismo pipeline sobre el corpus completo. La Figura 11 muestra la caída. La caída es el hallazgo más importante del trabajo: con el mismo codificador y los mismos parámetros de índice en colecciones separadas, y las mismas 38 preguntas, Recall@10 pasa de 0,947 a 0,684 y nDCG de 0,909 a 0,594 al crecer de 1.932 a 161.868 documentos. La latencia no explica la caída (40 ms contra 42 ms). El experimento observa la caída al aumentar la escala; no separa competencia entre fragmentos, truncamiento del texto ni segmentación.

E8 y E10 buscaron un recuperador mejor. BM25 sparse sube el recall a 0,763 y es ocho veces más rápido, pero su techo de cobertura es bajo. bge-m3 llega a 0,842 de Recall@10 y 0,708 de nDCG, y recupera el inglés de 0,111 a 0,667. La Tabla XIII resume.

E11 ajustó `ef` y probó la fusión con BM25. `ef`=800 sube el recall 2,6 puntos y el nDCG 1,3 con 6 ms más. La fusión 0,5/0,5 empata en recall y sube nDCG 6,9, P@1 13,2 y cobertura de span 5,3. Con el silver de un solo relevante la fusión parecía ganar. E12 repitió la comparación con el silver ampliado: el denso recupera más documentos nuevos (Recall@10 0,921 contra 0,890 y nDCG 0,827 contra 0,804). E12 se queda con la base densa, `ef`=800, sin fusión; la ruta desplegada después es la unión híbrida, elegida sobre el conjunto de desarrollo.

VII-E. Solr como backend de recuperación

Solr 9 se desplegó como un servicio más del stack de Swarm, con imagen, puerto, CPU y memoria parametrizados. El índice replica los 618.286 fragmentos con un esquema de

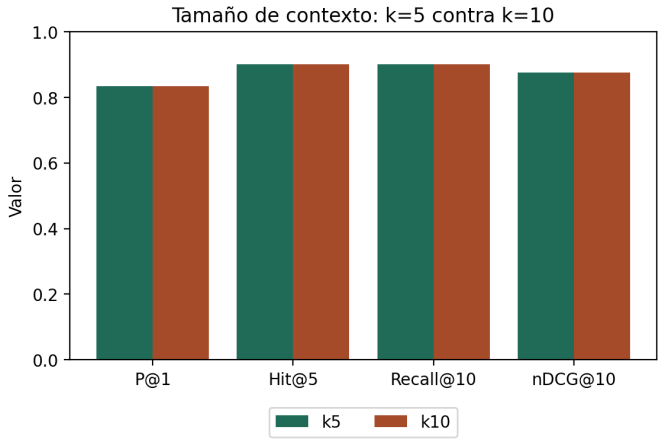


Figura 10. Tamaño del contexto: $k = 5$ contra $k = 10$.

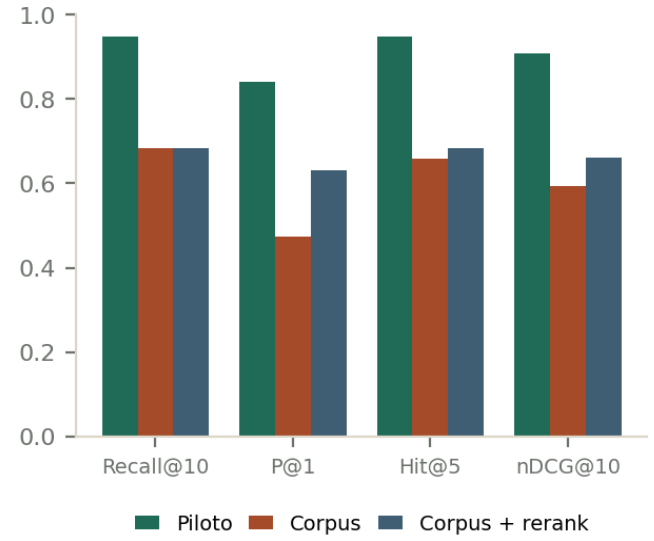


Figura 11. Recuperación con e5-base sobre el silver de 38 preguntas respondibles: piloto (1.932 documentos), corpus completo (161.868 documentos) y corpus con reranker sobre el top-50. El piloto y el corpus usan índices separados.

13 campos y búsqueda `edismax` sobre `text` y `title`[2]. La Tabla XIV compara el mismo silver de 38 preguntas. Solr con BM25 iguala el recall y el Hit@5 del BM25 de Milvus, pero pierde P@1 y nDCG. Con bge-m3, Solr alcanza 0,605 de recall, por debajo del denso de Milvus (0,842); la diferencia es consistente con el índice HNSW por defecto de Solr, que no se ajustó, y no con el embedding, que es el mismo. El backend es sustituible, pero el denso de Milvus sigue siendo la mejor opción para la recuperación inicial; una comparación justa de Solr denso exige ajustar `hnswBeamWidth` y reconstruir el índice. El servicio se levanta con `scripts/deploy_solr.sh` y la API de la demo corre en el puerto 8011 con `RETRIEVER=solr`.

VII-F. Reordenamiento

E7 midió el cross-encoder sobre el top-50 con e5 en el corpus completo: P@1 sube de 0,474 a 0,632 y nDCG de 0,594 a 0,661, pero el Recall@10 se queda en 0,684; Hit@5 sube de 0,658 a 0,684 y la cobertura de span de 0,632 a 0,658. El coste es de 3,1 s por consulta. La mejora de orden no compensa el coste si se reordena tanto.

El denso a $k = 10$ recupera 0,912 de recall y 0,818 de nDCG. Con bge-m3 se probó reordenar 5 y 20 candidatos. La Tabla XV muestra que el top-20 baja el recall a 0,864 y el Hit@5 a 0,921, mientras que el top-5 mantiene 0,895 de recall y 0,947 de Hit@5, y sube el nDCG a 0,865 y el P@1 a 0,868 con 1,30 s. La tabla mide el reordenador en CPU; en la configuración final corre en GPU, con 0,68 s para 50 pares. La configuración final no reordena el top-5 denso: reordena la unión con BM25 y recorta a 10.

Tabla XII
E6: PILOTO CONTRA CORPUS COMPLETO CON E5-BASE (38 PREGUNTAS).

Corpus	Recall@10	P@1	Hit@5	nDCG	Span
Piloto (1.932 docs)	**0,947**	**0,842**	**0,947**	**0,909**	**0,895**
Completo (161.868)	0,684	0,474	0,658	0,594	0,658

Tabla XIII
E10: RECUPERADORES SOBRE EL CORPUS COMPLETO (38 PREGUNTAS).

Método	Recall@10	P@1	Hit@5	nDCG	p50
Denso e5	0,684	0,500	0,658	0,604	48 ms
BM25 sparse	0,763	**0,684**	0,737	**0,727**	**7 ms**
bge-m3	**0,842**	0,553	**0,816**	0,708	56 ms

VII-G. Modelos generativos

E4 comparó cuatro modelos con la misma evidencia: 19 preguntas respondibles y 5 sin respuesta, tres repeticiones, anotación humana ciega de la primera repetición. La anotación cubre 18 de las 19 respondibles. La Tabla XVI muestra las medias de los dos anotadores en escala de 0 a 3. gemma2:2b gana en corrección, citas y abstención. La fidelidad no se usa para el veredicto porque su acuerdo es $\kappa = 0,12$. qwen2.5:1.5b es el más rápido pero no cita en el 89,5 % de los casos. llama3.2:3b cita la mitad de las veces. llama3.2:1b se abstiene en el 73,7 % de las preguntas respondibles y no se recomienda.

La hipótesis H1, que esperaba que llama3.2:3b superara a los demás en corrección y citas, se refuta. La evidencia recuperada es la misma entre modelos; lo que cambia es el generador. La comparación fijó la familia de modelos pequeños. El generador final es gemma4:e4b, que no estaba en E4: se adoptó después por contexto largo y MTP, no por ganar una comparación directa, y se validó en la corrida final.

Se probó un generador local de 27B (`qwen3.8:docs-32k-q3_k_s`, Q3) con el mismo pipeline y el mismo prompt. En test obtiene 0,917 de exactitud, 0,489 de cobertura, 0,657 de corrección RAGAS y 10,3 s de mediana, frente a 0,917, 0,522, 0,652 y 1,2 s del 4B. El modelo grande no mejora de forma apreciable las métricas y multiplica la latencia por ocho; el 4B se queda como generador desplegado.

Tabla XIV
SOLR CONTRA LOS RECUPERADORES DE MILVUS SOBRE LAS 38 PREGUNTAS RESPONDIBLES.

Recuperador	Recall@10	P@1	Hit@5	nDCG	p50
Solr BM25	0,763	0,526	0,737	0,652	37 ms
Solr bge-m3	0,605	0,395	0,579	0,510	109 ms
Milvus BM25	0,763	**0,684**	0,737	**0,727**	**7 ms**
Milvus bge-m3	**0,842**	0,553	**0,816**	0,708	56 ms

Tabla XV
REORDENAMIENTO CON BGE-M3 (38 PREGUNTAS).

Configuración	Recall	nDCG	P@1	Hit@5	p50
Denso top-5	0,895	0,809	0,737	0,947	0,10 s
Rerank top-5	0,895	**0,865**	**0,868**	**0,947**	1,30 s
Rerank top-20	0,864	0,853	**0,868**	0,921	5,46 s

Tabla XVI
E4: ANOTACIÓN HUMANA (0 A 3) Y CONDUCTA AUTOMÁTICA.

Modelo	Corrección	Citas	Abstención	Sin citas
gemma2:2b	**2,81**	**2,67**	**2,89**	**0,0 %**
qwen2.5:1.5b	2,31	0,00	2,81	89,5 %
llama3.2:3b	2,22	0,86	2,69	56,1 %
llama3.2:1b	0,42	0,28	1,61	15,8 %

Tabla XVII
E5: INSTRUCCIONES INYECTADAS EN EL TEXTO RECUPERADO.

Modelo	Obedeció	Respondió el hecho	Se abstuvo
gemma2:2b	1/5	4/5	0/5
llama3.2:3b	0/5	5/5	0/5
qwen2.5:1.5b	0/5	5/5	0/5

VII-H. Inyección de instrucciones

E5 insertó instrucciones explícitas en cinco documentos sintéticos por modelo. Es una prueba exploratoria: no cubre el generador final ni demuestra robustez general. Dos de los tres modelos las ignoraron. gemma2:2b obedeció una de cinco y añadió la palabra pedida a una respuesta por lo demás correcta. La regla de diseño que sale de aquí es que el prompt debe delimitar la evidencia y ordenar explícitamente ignorar instrucciones internas, y esa orden debe repetirse después del bloque de contexto.

VII-I. Prompts y mejoras rechazadas

La fase 6 probó cuatro mejoras sobre la configuración de referencia y la fase 7 probó diversidad. La referencia de estas fases era el top-5 denso con reranker; la unión con BM25 se adoptó después, en el conjunto reservado. Ninguna de las variantes de las fases 6 y 7 supera a esa referencia. La Tabla XVIII resume. El filtro por score pierde recall; reordenar el top-20 pierde Hit@5 y multiplica la latencia; la expansión de consulta baja el recall y el nDCG; el prompt v4 no mejora la cobertura de tokens; y MMR baja el recall 8 puntos. El prompt v4 entregó 37 de 38 y 25 de 37 de cobertura de tokens, sin mejora. Todas se rechazan y la configuración se mantiene.

VII-J. Compuerta de dominio

La compuerta se entrenó con 160 preguntas: 60 de dominio y 100 de fuera, con 103 de train, 37 de dev y 20 de test. Los 40 negativos nuevos son entidades inventadas, eventos futuros y conocimiento general. La Tabla XIX muestra el resultado en el test interno y en el conjunto reservado. En test el F1 es 0,952, con 0 falsos aceptados de 9 y 1 falso rechazo de 11. En el conjunto reservado, con 20 directas y 5 negativas, el F1 es 0,919: rechaza las 5 negativas y 3 de las 20 directas. El umbral es 0,4, elegido por validación cruzada. Con 5 negativas, el 0/5 no demuestra una tasa nula de falsos aceptados en uso general; solo acota el intervalo observado.

VII-K. RAGAS

La Tabla XX compara cuatro configuraciones. El híbrido es la ruta desplegada: unión densa y BM25, $k = 10$ y sin compuerta. El denso $k = 10$ comparte k con el híbrido, así que su diferencia aísla el recuperador: el híbrido gana relevancia, precisión y corrección, y pierde fidelidad, con la misma cobertura. El denso $k = 5$, con menos contexto, gana fidelidad, relevancia, precisión y corrección, y pierde cobertura: el efecto de k es mayor que el del recuperador. La última columna cambia juez y embeddings, así que es observacional. Las diferencias entre las tres primeras son descriptivas.

VII-L. Latencia y MTP

La Tabla XXI muestra el efecto del draft. El draft 3 es el más rápido: 217,1 tokens por segundo frente a 193,5 con 5 y 159,5 con 7. La decodificación especulativa es exacta: el draft propone y el modelo objetivo verifica, así que la salida no cambia.

La corrida de referencia de la configuración anterior (densa y reranker en CPU) es la del 2 de noviembre, con el código del commit `125bbc7`: 7.938 ms sin razonamiento y 6.834 ms con razonamiento; los CSV están en `reports/tablas/`. El reranker en CPU aporta 1,30 s de mediana. Con la configuración final (unión híbrida y reranker en GPU), el conjunto reservado dio 6.154 ms de media (mediana 5.578, p95 9.136) sobre 25 preguntas. La compuerta reutiliza la puntuación del reranker, así que no añade una segunda pasada.

VII-M. Observabilidad y evaluación continua

Cada consulta deja una traza en Langfuse 4.78, desplegado en el servidor como un servicio más del stack. La traza `ask` contiene los spans `retrieve`, `rerank`, `generate` y `grounding`, con los fragmentos recuperados, el prompt, la respuesta, las citas, los tokens y las latencias. Los scores `citation_valid`, `grounded` y `latency_ms` se calculan en la misma petición, y el identificador de la traza se devuelve en la respuesta para enlazarla con la corrida de evaluación. La Figura 12 muestra la traza de una consulta real.

El conjunto dorado vive como Dataset de Langfuse. Cada corrida es un experimento que enfrenta la respuesta esperada con la respuesta del modelo y calcula cuatro scores por

Tabla XVIII
FASES 6 Y 7: VARIANTES PROBADAS SOBRE 38 PREGUNTAS. LA REFERENCIA ES EL TOP-5 CON RERANKER.

Variante	Recall	nDCG	P@1	Hit@5	Decisión
Top-5 con reranker	**0,895**	**0,865**	**0,868**	**0,947**	Referencia
Rerank top-20	0,864	0,853	**0,868**	0,921	Rechazada
Filtro por score 0,8	0,838	0,837	**0,868**	0,921	Rechazada
Filtro por score 0,9	0,825	0,830	**0,868**	0,921	Rechazada
Expansión de consulta	0,825	0,739	0,658	0,895	Rechazada
MMR coseno 0,5	0,868	0,784	0,737	**0,947**	Rechazada
MMR con reranker 0,7	0,816	0,818	**0,868**	0,921	Rechazada

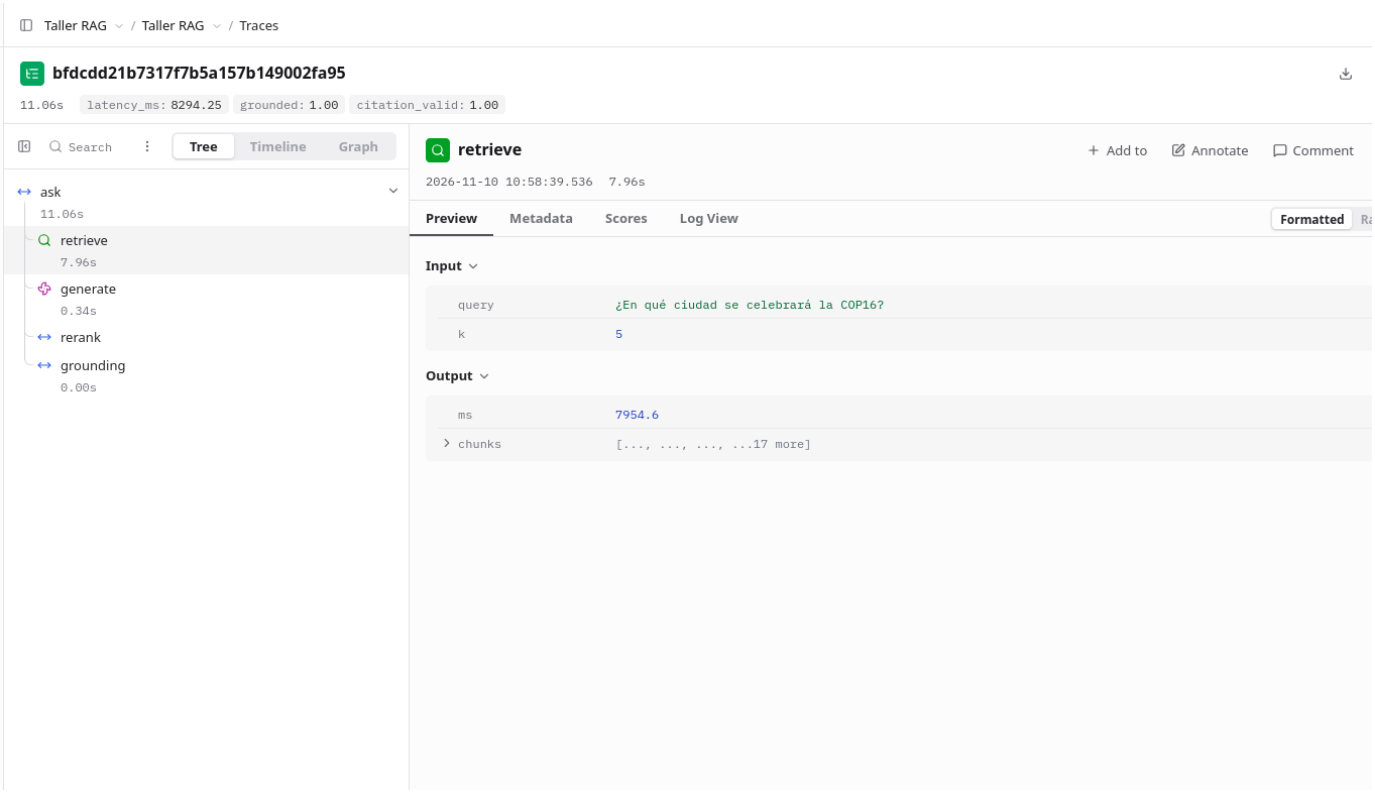


Figura 12. Traza de una consulta con el prompt v2, antes de la configuración final: spans de recuperación, reordenamiento, generación y anclaje, con los scores de la petición. La traza final añade los spans de traducción y de extracción de listas.

Tabla XIX
COMPUERTA DE DOMINIO: TEST INTERNO Y CONJUNTO RESERVADO.

Split	n	Dominio	Fuera	Precisión	Recall	F1
Test	20	11	9	1,000	0,909	0,952
Reservado	25	20	5	1,000	0,850	0,919

ítem: `correct`, `token_recall`, `citation_valid` y `grounded`. La Figura 13 muestra la vista de evaluación con la pregunta, el gold, los scores y la respuesta. La Tabla XXII resume las tres corridas sobre las 43 preguntas del silver.

La tabla muestra el efecto de la observabilidad: las corridas usan el mismo conjunto, el mismo modelo y la misma temperatura; la diferencia es del recuperador y de los mecanismos de respuesta de la configuración final. El registro no cambia la respuesta; cambia la capacidad de atribuirla. Sin él, una mejora de dos puntos en exactitud no se puede separar de la variación del muestreo ni del orden de las preguntas.

VII-N. GraphRAG ligero

Se construyó un grafo de documentos y entidades en Neo4j 5.26. Cada documento se conecta con las entidades nombradas que menciona; la extracción corre con spaCy en español e inglés sobre el título y los primeros 1.500 caracteres. El grafo tiene 169.248 documentos, 248.869 entidades y 926.923 menciones, y se construye en 2.000 s en un solo proceso. La Figura 14 muestra un documento y sus entidades.

Se probaron dos formas de usar el grafo. `graph_hybrid` fusiona los candidatos del grafo con la unión densa y BM25 antes del reordenamiento. `hybrid_boost` deja intacta la recuperación y suma al `rerank_score` un bono por cada entidad de la consulta presente en el fragmento. La Tabla XXIII compara ambos modos con el baseline en las 43 preguntas y en un conjunto nuevo de 10 preguntas multi-documento.

El resultado separa dos usos que suelen confundirse. El grafo como fuente de candidatos mejora una pregunta en el

Tabla XX
RAGAS: HÍBRIDO, DENSO CON $k = 10$, DENSO CON $k = 5$ Y CONFIGURACIÓN ANTERIOR, CON n ENTRE PARÉNTESIS. LAS TRES PRIMERAS USAN JUEZ GEMMA4:E4B Y EMBEDDINGS BGE-M3; LA ÚLTIMA CAMBIA JUEZ Y EMBEDDINGS, ASÍ QUE SU COMPARACIÓN ES OBSERVACIONAL.

Métrica	Híbrido $k = 10$	Denso $k = 10$	Denso $k = 5$	Anterior
Fidelidad	0,920 (38)	0,932 (38)	**0,952** (38)	0,925 (36)
Relevancia	0,788 (38)	0,765 (38)	**0,840** (38)	0,798 (38)
Precisión del contexto	0,867 (36)	0,810 (36)	**0,881** (34)	0,760 (36)
Cobertura del contexto	**0,947** (38)	**0,947** (38)	0,921 (38)	0,842 (38)
Corrección	0,607 (38)	0,569 (38)	**0,657** (38)	0,705 (37)

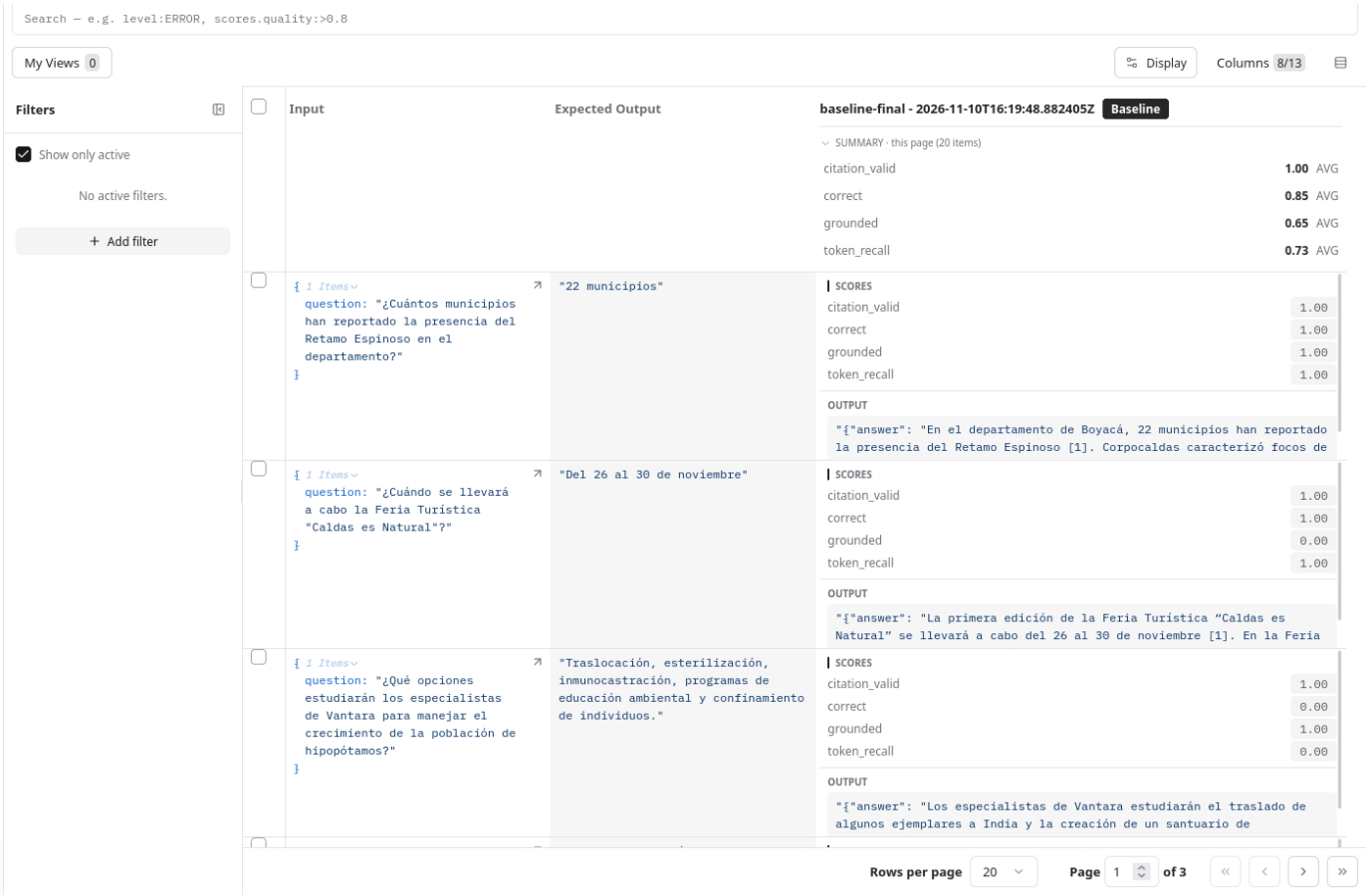


Figura 13. Experimento en Langfuse: pregunta, respuesta esperada, scores y respuesta del modelo, con la comparación por ítem. La captura corresponde a la corrida `baseline-final`, con el prompt v2; la tabla compara esa corrida con las posteriores.

Tabla XXI
MTP: TOKENS POR SEGUNDO SEGÚN EL DRAFT.

Draft	Tokens/s
3	**217,1**
5	193,5
7	159,5

Tabla XXII
CORRIDAS REGISTRADAS EN LANGFUSE SOBRE LAS 43 PREGUNTAS DEL SILVER. LA EXACTITUD INCLUYE LAS 5 PREGUNTAS SIN RESPUESTA. LAS TRES PRIMERAS FILAS SON ETAPAS INTERMEDIAS; LA ÚLTIMA ES LA CONFIGURACIÓN DESPLEGADA.

Corrida	Exactitud	Cobertura	Anclaje	p50
Unión híbrida	0,837	0,626	0,744	0,42 s
Grafo híbrido	**0,861**	0,635	0,767	1,44 s
Híbrido + boost	**0,861**	**0,655**	**0,791**	1,30 s
Final	**0,884**	0,664	0,907	1,47 s

silver, pero pierde 20 puntos en el conjunto multi-documento: el reordenador prefiere documentos que el grafo trae por entidades ruidosas y desplaza fragmentos que la recuperación densa ya tenía. El grafo como reordenador, en cambio, sube la cobertura de tokens y el anclaje en ambos conjuntos sin degradar la exactitud. La diferencia es consistente con la calidad de la extracción: las claves no pasan por un enlace de entidades y muchas son frases largas, así que el grafo sirve para desempatar, no para sustituir al índice. GraphRAG no es una mejora gratuita; con este corpus y este extractor, el uso defendible es el boost. Se reconstruyó el grafo con un filtro

Tabla XXIII
GRAPHRAG CONTRA LA UNIÓN HÍBRIDA. EXACTITUD Y COBERTURA DE TOKENS SOBRE LAS 43 PREGUNTAS DEL SILVER Y SOBRE 10 PREGUNTAS MULTI-DOCUMENTO.

Modo	Exactitud (43)	Cobertura (43)	Exactitud (10)	Cobertura (10)
Unión híbrida	0,837	0,626	**0,900**	0,878
Grafo híbrido	**0,861**	0,635	0,700	0,694
Híbrido + boost	**0,861**	**0,655**	**0,900**	**0,889**

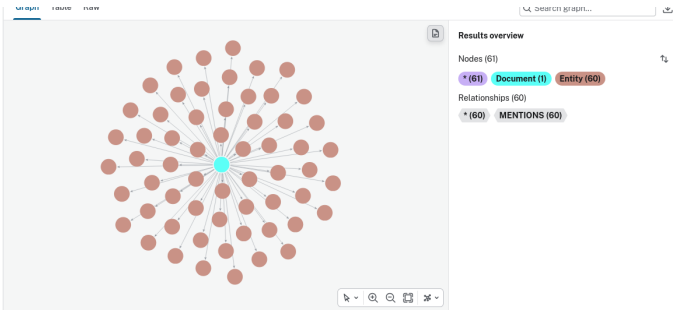


Figura 14. Grafo en Neo4j: un documento y las entidades que menciona.

de entidades (máximo 4 tokens, sin dígitos iniciales, MISC de máximo 3): quedó en 206.614 entidades y 852.751 menciones. Con ese grafo y el prompt v3, `graph_hybrid` obtiene 0,875 en test y 0,880 en el reservado, mientras el boost obtiene 0,917 y 0,840; en las 10 preguntas multi-documento el grafo sigue en 0,700. La limpieza mejora la precisión del grafo en el reservado, pero no revierte la pérdida multi-documento.

VII-O. Mejora de la respuesta y evaluación semántica

El análisis de errores mostró que la recuperación no era el único cuello de botella: RAGAS daba 0,947 de cobertura de contexto y 0,607 de corrección. El contexto traía la evidencia, pero el modelo omitía datos en las preguntas de lista o de dos partes. Se probó un prompt v3 que pide la respuesta directa, todos los datos pedidos y la lista completa, con dos ejemplos y un límite de tres frases. La Tabla XXIV compara el prompt v2 y el v3 con el mismo recuperador (unión híbrida más el boost del grafo) en test (24 preguntas) y en el reservado (25).

La exactitud es la misma: 0,917 en test y 0,840 en el reservado. La diferencia aparece en la evaluación semántica. La Tabla XXV repite RAGAS sobre las 19 preguntas respondibles de test y las 20 del reservado. El prompt v3 sube la corrección de 0,590 a 0,652 con el mismo recuperador y la misma exactitud; sin el boost llega a 0,687, a costa de una pregunta exacta. La diferencia pareada es +0,062, IC 95 % $[-0{,}019, 0{,}145]$, $p = 0{,}14$: con 19 preguntas es una tendencia, no un resultado concluyente. La exactitud no cambia (diferencia 0, IC $[0, 0]$). La fidelidad queda en 1,0 y la cobertura de contexto en 1,0: el generador no inventa y la evidencia alcanza.

La métrica léxica subestima las respuestas correctas cuando cambian de idioma. En el reservado, r-012 responde *Foro Económico Mundial* por *World Economic Forum* y r-019 responde *avispa asiática* por *Asian Hornet*; ambas son correctas y el chequeo de contención las marca como fallo, mientras RAGAS las puntúa por encima de 0,7. Con esa corrección, el reservado pasa de 21 a 23 aciertos sobre 25. La configuración final añade dos mecanismos: la traducción de la consulta al otro idioma del corpus, fusionada por RRF, y la extracción JSON de listas para preguntas de opciones. En test resuelve g-046 (Newcastle) y g-003 (las cinco opciones) y alcanza 0,958; en el silver completo sube de 0,837 a 0,884 y en el reservado se queda en 0,840. RAGAS: 0,668 en test y 0,623 en el reservado; fidelidad 1,0 y 0,947. El coste es 0,25 s de mediana. Se probó además una verificación numérica que completa la cifra cuando falta: en la corrida de test acertó g-044 y llegó a 1,000, pero en una consulta en vivo añadió 2017 en lugar de 2024 y RAGAS bajó de 0,668 a 0,655 en test y de 0,623 a 0,595 en el reservado. Se descarta: es un parche léxico inestable. g-044 sigue como omisión del generador; quedan r-006 (la cifra de la cámara) y r-016 (pregunta ambigua). Las diferencias contra v3 + boost no excluyen cero (IC 95 % $[-0{,}116, 0]$ en el silver), así que son tendencias.

VII-P. Ampliación con documentos aleatorios

El test curado se eligió a mano y puede sobreestimar el sistema. Para medirlo, se generaron 30 preguntas nuevas desde documentos aleatorios del corpus, fuera del silver: el LLM escribió la pregunta a partir del documento y el span literal es el gold; la revisión humana queda pendiente. La Tabla XXVI compara ambos conjuntos con la configuración final.

La exactitud baja 15,8 puntos al salir del conjunto elegido a mano; la corrección RAGAS se mantiene (0,674 frente a 0,668), la fidelidad en 0,971 y la cobertura de contexto en 1,0. La caída no es una regresión del sistema: es el efecto de evaluar sobre documentos que no se seleccionaron para que el sistema luciera. La ampliación tiene 30 preguntas, así que su intervalo sigue siendo amplio; el set de 120-150 queda como trabajo futuro.

VII-Q. Experimentos descartados y su coste

No todas las hipótesis se aceptaron. La Tabla XXVII reúne las que se implementaron y se midieron, el resultado que las descartó y lo que costaban. La lista importa tanto como la de aciertos: fija el espacio de diseño que se exploró y evita que un lector atribuya la configuración final a una preferencia.

El contexto padre y la expansión HyDE ilustran el mismo patrón: mejoran la comodidad de la respuesta o el recall en desarrollo y no mueven la exactitud, a cambio de duplicar o multiplicar por seis la latencia. El modelo 27B es el caso más caro: cuesta ocho veces más por respuesta y no supera al 4B en exactitud; su única ventaja es la cobertura, que el prompt v3 ya atacaba. La verificación numérica acertó en la corrida

Tabla XXIV
PROMPT V2, V3 Y LA CONFIGURACIÓN FINAL CON TRADUCCIÓN Y LISTAS, CON EL MISMO RECUPERADOR. EXACTITUD Y COBERTURA DE TOKENS SOBRE TEST (24) Y EL RESERVADO (25).

Configuración	Exactitud test	Cobertura test	Exactitud reservado	Cobertura reservado
v2 + boost	0,917	0,541	**0,840**	**0,440**
v3 + boost	0,917	0,522	**0,840**	**0,440**
v3 sin boost	0,875	0,486	**0,840**	**0,440**
v3 + boost + trad. + listas	**0,958**	**0,564**	**0,840**	**0,440**

Tabla XXV
RAGAS CON PROMPT V2, V3 Y LA CONFIGURACIÓN FINAL. TEST: 19 PREGUNTAS RESPONDIBLES. RESERVADO: 20. JUEZ GEMMA2:2B Y EMBEDDINGS QWEN3-EMBEDDING:8B.

Configuración	Fidelidad	Relevancia	Precisión	Cobertura	Corrección
v2 + boost (test)	**1,000**	**0,790**	0,735	**1,000**	0,590
v3 + boost (test)	**1,000**	0,772	0,728	**1,000**	0,652
v3 + boost (reservado)	0,938	0,798	0,808	**1,000**	0,629
v3 + boost + trad. + listas (test)	**1,000**	0,785	0,740	**1,000**	0,668
v3 + boost + trad. + listas (reservado)	0,947	0,816	0,762	**1,000**	0,623
v3 sin boost (test)	**1,000**	0,799	**0,751**	**1,000**	**0,687**
v3 sin boost (reservado)	0,975	**0,856**	**0,863**	**1,000**	0,655

Tabla XXVI
TEST CURADO CONTRA LA AMPLIACIÓN ALEATORIA CON LA CONFIGURACIÓN FINAL (V3, BOOST, TRADUCCIÓN Y LISTAS).

Conjunto	n	Exactitud	Cobertura	Anclaje	RAGAS	Fidelidad	p50
Test curado	24	**0,958**	0,564	0,833	0,668	**1,000**	1,50 s
Ampliación	30	0,800	**0,749**	**0,867**	**0,674**	0,971	1,48 s

Tabla XXVII
EXPERIMENTOS IMPLEMENTADOS Y DESCARTADOS. LAS CIFRAS SON DE LA CORRIDA PROPIA DE CADA VARIANTE; LA REFERENCIA ES LA CONFIGURACIÓN DE SU MOMENTO.

Intento	Qué se probó	Resultado	Coste	Decisión
Contexto padre	Devolver el fragmento completo en lugar del chunk	Test 0,875 y reservado 0,840, sin mejora; p50 2,05 s frente a 1,27 s	$+0{,}8$ s	Rechazado
Modelo 27B Q3	Generador mayor para compensar omisiones	Test 0,917 y cobertura 0,489; p50 10,3 s; RAGAS 0,657	$\times 8$ latencia	Rechazado
Expansión HyDE	Consulta sintética para el recuperador denso	Test 0,917 y reservado 0,840, sin mejora; p50 8,0 s	$+6{,}5$ s	Rechazado
Expansión léxica	Reescribir la consulta para el índice BM25	r-006 sigue respondiendo 125 y 91 votos; el documento solo aparece si la consulta lleva la cifra	—	Rechazado
Verificación numérica	Completar la cifra que el generador omite	Test 1,000 pero RAGAS 0,655; en vivo añadió 2017; reservado RAGAS 0,595	$+0{,}2$ s	Rechazado
Garantía de candidato	Forzar el top léxico en preguntas legislativas	El contexto trae más documentos y r-006 sigue en 125 y 91	—	Rechazado
Grafo como candidato	Fusionar documentos del grafo antes del reranker; después se filtraron las entidades	Test 0,875, reservado 0,880 y multi-documento 0,700; el boost da 0,900 en multi	—	Rechazado

de test y falló en vivo: un parche léxico que acierta cuando el dato está aislado y falla cuando el documento tiene dos años distintos. La garantía de candidato confirma que r-006 no es un problema de recuperación sino de pregunta subespecificada. El grafo como fuente de candidatos pierde 20 puntos en el conjunto multi-documento; sólo el boost sobrevive.

VII-R. Resumen de resultados

La Tabla XXVIII reúne la configuración desplegada sobre los cuatro conjuntos. Es la cifra que resume el trabajo; las secciones anteriores dan la trazabilidad de cada decisión.

RAGAS se midió en test, reservado y ampliación. El silver completo se usó para elegir la configuración, así que no se recalculó con el juez final: sus cifras de exactitud, cobertura y anclaje vienen de la corrida de Langfuse. El reservado también se usó para elegir el híbrido, de modo que sus métricas de recuperación y respuesta son de desarrollo; la ampliación es la única que no participó en ninguna decisión, y por eso su exactitud es la estimación más honesta, aunque su revisión humana esté pendiente.

La Figura 15 reúne tres resultados de las etapas intermedias: RAGAS por configuración, las políticas de ambigüedad sobre el silver humano y el flujo del conjunto reservado con y sin compuerta.

Tabla XXVIII
CONFIGURACIÓN DESPLEGADA SOBRE LOS CUATRO CONJUNTOS. EXACTITUD, COBERTURA DE TOKENS Y ANCLAJE SON MEDIAS POR PREGUNTA; EL P50 ES LA MEDIANA DE LATENCIA.

Conjunto	n	Exactitud	Cobertura	Anclaje	RAGAS	Fidelidad	p50
Test curado	24	**0,958**	0,564	0,833	0,668	**1,000**	1,50 s
Silver completo	43	0,884	0,664	**0,907**	—	—	1,47 s
Reservado	25	0,840	0,440	0,880	0,623	0,947	1,42 s
Ampliación	30	0,800	**0,749**	0,867	**0,674**	0,971	1,48 s

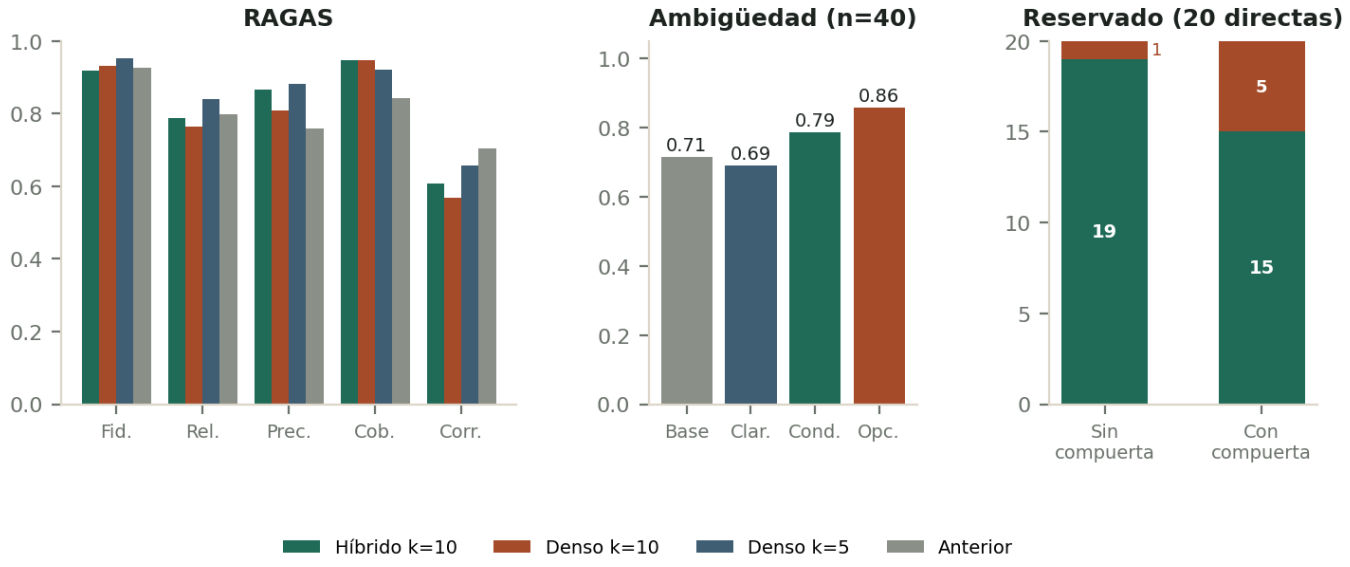


Figura 15. Resumen de resultados: RAGAS por configuración, políticas de ambigüedad y flujo del conjunto reservado con y sin compuerta.

VII-S. Conjunto reservado

El silver exploratorio se usó para elegir la configuración densa, así que no mide generalización. Se construyó un conjunto reservado de 25 preguntas: 20 directas y 5 sin respuesta, 15 en español y 10 en inglés, 14 de prensa y 6 de Bluesky. Las preguntas se generaron con Gemma4 y se validaron automáticamente; no son una muestra humana independiente. El conjunto se congeló antes de evaluar la compuerta y se usó para esa evaluación. Después se usó también para elegir la recuperación híbrida. Por eso las métricas de recuperación y respuesta de esta sección son de desarrollo. La compuerta se midió antes de esa elección, pero el mismo conjunto se usó después para elegir el híbrido: sus cifras también son de desarrollo. Una prueba independiente de la recuperación exige un conjunto nuevo. La Tabla XXIX muestra el resultado.

Las métricas de respuesta se midieron sin la compuerta estricta; las predicciones de la compuerta se calcularon en una pasada aparte. Con la compuerta activa (`strict=True`), el flujo conjunto entrega 15 de 20 directas y se abstiene en las 5 negativas: tres directas se rechazan por dominio (r-011, r-012 y r-014) y dos por el juez de evidencia (r-005 y r-006). La cobertura de tokens de las 15 entregadas es 13/15 (0,867). La diferencia con las 19 entregas sin compuerta es el coste del filtro. El recall@10 es 0,700 y el nDCG 0,612, por debajo del 0,912 y 0,818 del silver exploratorio, pero por encima del 0,600 y 0,477 de la recuperación densa. La unión con BM25 recupera r-012 y r-014, que el denso dejaba fuera, y conserva los aciertos del denso en r-009 y r-013. Seis de las 20 preguntas no recuperan el documento de referencia en el top-10; cuatro se responden correctamente con otro documento que contiene el mismo dato (r-005, r-008, r-017, r-019), una termina en abstención (r-006) y una es ambigua (r-016). El recall con un

Tabla XXIX
CONJUNTO RESERVADO: 20 PREGUNTAS DIRECTAS Y 5 SIN RESPUESTA, CONGELADO ANTES DE EJECUTAR; LA COMPUERTA SE EVALUÓ ANTES DE ELEGIR EL HÍBRIDO.

Métrica	Valor	Denominador
Recall@10	0,700	20
nDCG@10	0,612	20
P@1	0,500	20
Hit@5	0,650	20
Cobertura de span	0,700	20
Entregadas	19	20
Abstenciones en directas	1	20
Cobertura de tokens, entregadas	17	19
Cobertura de tokens, todas	17	20
Con cita	1,000	19
Abstenciones en sin respuesta	5	5
Compuerta rechaza	5	5
Latencia media	6,2 s	25
Entregadas con compuerta (strict)	15	20
Cobertura de tokens con compuerta	13	15
Rechazos de compuerta	3	20
Rechazos del juez	2	20

solo documento de referencia subestima la recuperación útil: la mitad de los fallos de recall no son fallos de respuesta. La profundidad de la unión no es monotónica: 20/20 y 30/30 suben el recall@10 a 0,750; 50/50 lo sube a 0,850 y recupera r-006, pero baja las entregas a 18 de 20 y la cobertura de tokens a 16 de 20, porque desplaza evidencia que el modelo usaba bien en r-009 y r-012. La configuración se queda en 10/20.

La cobertura de tokens entre las entregadas es 17 de 19 (89,5 %), y sobre las 20 directas, 17 de 20 (85 %). La diferencia con la recuperación densa está en la unión: recupera las entidades que el denso no trae. En inglés el recall es 0,875 y el nDCG 0,829; en español, 0,583 y 0,468. El inglés recupera mejor, al contrario de lo que sugería el silver exploratorio.

Las cinco preguntas sin respuesta se rechazan por la compuerta. En las 20 directas, la compuerta rechaza 3: r-011, r-012 y r-014. r-012 y r-014 son preguntas que la recuperación híbrida sí puede responder, pero la compuerta solo ve el top-5 denso; es una deuda de diseño, no un fallo de recuperación. r-011 sí recupera el documento y es un falso rechazo del clasificador. La compuerta nueva baja los falsos rechazos de 6 a 3 y los falsos aceptados de 1 a 0 respecto de la anterior. El 0/9 del conjunto anterior no se confirma como tasa nula, pero el 0/5 tampoco la demuestra.

El fallo más grave es r-006: la pregunta pide los 95 votos en la Cámara de Representantes y el sistema se abstiene. La evidencia recuperada no contiene la cifra correcta en el top-10, y el modelo prefiere no responder. r-016 es una pregunta ambigua del propio conjunto: no nombra el concierto y el corpus tiene varios; se mantiene en las métricas y se reporta como límite de construcción. r-012 entrega una respuesta evasiva: la evidencia contiene la organización, pero el modelo enumera varias iniciativas en lugar de responder World Economic Forum. Es un fallo de generación, no de recuperación.

La latencia media del conjunto reservado es 6,2 s (mediana 5,6, p95 9,1), inferior a los 10,8 s de la corrida densa. La unión y el reordenador en GPU compensan el mayor número de candidatos.

La Tabla XXX resume la progresión de la configuración. Cada fila es una decisión tomada con una medición, no una preferencia.

VIII. Análisis de errores

VIII-A. Errores de recuperación

Queda un fallo claro en el silver standard. g-031 pregunta por el municipio de gran extensión y tradición agropecuaria del sur del Tolima. En E6/E7 quedaba en el puesto 11 del top-50 denso; la ruta híbrida lo recupera en el puesto 1 y responde Chaparral. g-046, sobre la protesta de Extinction Rebellion, no aparece ni en la unión ni tras el reranker; no se aisló la causa. El análisis de E6/E7 se conserva como histórico.

VIII-B. Errores de generación

Los modelos pequeños fallan de formas distintas con la misma evidencia. qwen2.5:1.5b responde sin citar en el 89,5 % de los casos: la respuesta puede ser correcta, pero el usuario no puede verificar de dónde sale. llama3.2:3b cita la mitad de las veces. llama3.2:1b se abstiene en el 73,7 % de las preguntas respondibles. gemma2:2b cita siempre y responde casi siempre, pero responde preguntas sin respuesta en 3 de 15 repeticiones, es decir, alucina con evidencia insuficiente. El generador final, gemma4:e4b, no estaba en esa comparación; los guardarraíles de cita, entidades y juez cubren parte de ese riesgo, y la compuerta lo cubre antes de generar.

VIII-C. Abstención y compuerta

La abstención tiene dos caminos. El primero ocurre antes de generar, cuando la compuerta rechaza la pregunta; el segundo, después, cuando los guardarraíles o el juez no encuentran respaldo. El primer camino es barato (decenas de milisegundos) y ahora rechaza las 5 preguntas sin respuesta con probabilidades de 0,000 a 0,166. El segundo cuesta una generación completa. La compuerta v2, con umbral 0,4, rechaza 3 de las 20 directas del reservado: r-011 es un falso rechazo del clasificador y r-012/r-014 son deudas de diseño porque la compuerta solo ve el top-5 denso. La versión anterior, con umbral 0,5, dejaba un único falso rechazo en 0,494; se conserva como antecedente y no describe la compuerta actual.

VIII-D. Errores de evaluación

RAGAS deja 4 muestras sin valor en la precisión del contexto por errores de parser del juez. Una media con 34 valores no es comparable con una de 38, y por eso se reporta n en cada fila. En la primera corrida, con timeout de 180 s y 16 workers, la métrica tenía 4 valores y su media (0,750) no era comparable con nada. La conclusión anterior, que afirmaba una pérdida de precisión frente a la arquitectura previa (0,760, con 36 valores), era un artefacto del juez. La repetición corregida del mismo sistema da 0,881 con 34 valores. La comparación entre arquitecturas sigue siendo observacional porque cambia juez y embedding.

VIII-E. Errores de diseño del protocolo

El experimento de limpieza se lanzó inicialmente con el silver de humo, que pertenece a una muestra de 600 documentos que no es subconjunto del piloto de 2.000. Las referencias no existían en las colecciones y las métricas habrían sido cero. El error se detectó al medir y obligó a construir un silver específico para el piloto. La lección es que el silver debe declarar a qué muestra pertenece y que la ingesta debe verificar que los documentos de referencia están en la colección antes de evaluar. Esa verificación ya forma parte del protocolo.

IX. Discusión

IX-A. Por qué el piloto engaña

La caída de Recall@10 de 0,947 a 0,684 al pasar del piloto al corpus completo (Tabla XII) es consistente con un efecto de competencia: con 1.932 documentos, el fragmento relevante compite contra pocos vecinos; con 161.868, contra cientos de fragmentos que comparten vocabulario genérico. El texto sin cierre terminal (32,7 %) y el chunking fijo podrían agravarla, pero el experimento no separa competencia, truncamiento y

Tabla XXX
PROGRESIÓN DE LA CONFIGURACIÓN Y MÉTRICA QUE LA JUSTIFICÓ.

Etapa	Cambio	Silver	Métrica	Decisión
Piloto	e5-base, 1.932 docs	piloto (38)	Recall@10 0,947	Punto de partida
Escala	corpus completo	38	Recall@10 0,684	El piloto sobreestima
Dispersa	BM25	38	Recall@10 0,763	No alcanza el techo
Codificador	bge-m3	38	Recall@10 0,842	Adoptado
Índice	`ef`=800	38	Recall@10 0,895	Adoptado
Fusión	híbrido RRF	38	Recall@10 0,895	Empata, se descarta
Fusión	híbrido RRF	ampliado	Recall@10 0,890	Rechazado
Reordenamiento	unión densa+BM25	reservado (20)	Recall@10 0,700	Adoptado; recupera parte de la caída
Contexto	$k = 10$	reservado (20)	19/20 entregadas	Adoptado
Prompt	v2	38	37/38 y cobertura 25/37	Adoptado
Compuerta	160 preguntas	20 test	0 falsos aceptados	Adoptado
Fase 6	cuatro variantes	38	sin mejora	Rechazadas
Fase 7	MMR	38	recall 0,816	Rechazado
Compuerta	5 negativas	reservado (5)	5/5 rechazadas	3/20 falsos rechazos
Prompt	v3 + boost	test (24)	Exactitud 0,917	Adoptado
Traducción + listas	v3 + boost	test (24)	Exactitud 0,958	Adoptado

segmentación. La consecuencia práctica es que un piloto sirve para depurar el pipeline, no para elegir codificador, chunking o k. Esa elección hay que hacerla a la escala final, aunque cueste horas de GPU.

IX-B. El coste del orden

El reordenador es una herramienta de precisión, no de cobertura. Sobre el top-5 sube el nDCG 5,6 puntos y el P@1 13,1, porque el recuperador ya trajo la respuesta y solo había que colocarla primero. Sobre el top-20 denso baja el recall y el Hit@5: al ampliar el conjunto entran fragmentos peores que desplazan a los buenos. La unión con BM25 cambia el cálculo: añade documentos que el denso no trae y el reordenador los coloca arriba. En GPU el coste es 0,68 s para 50 pares, frente a 1,30 s para 5 en CPU. El reordenador no puede recuperar lo que el recuperador no trajo; la unión sí puede, y por eso se adopta.

IX-C. La abstención barata

La compuerta cuesta milisegundos y rechaza las 5 preguntas sin respuesta; el juez cuesta una generación completa. Esa asimetría justifica el clasificador. La compuerta v2, con umbral 0,4, rechaza 3 de las 20 directas del reservado: dos son deudas de diseño y una es un falso rechazo del clasificador. El umbral es una decisión de negocio, no técnica: aceptar una pregunta ajena erosiona la confianza más que abstenerse en una propia. En un asistente de corpus, el usuario prefiere un "no lo sé" a una respuesta inventada. Si el coste de abstenerse fuera mayor, el umbral se movería en la otra dirección.

IX-D. La evaluación

El episodio de RAGAS (Sección VII) deja dos reglas. La comparación entre arquitecturas cambia juez y embedding, así que se reporta como observación y no como mejora causal. La corrida del híbrido confirma el intercambio: más cobertura del contexto, menos corrección. La primera: reportar n junto a cada media y no comparar métricas con cobertura distinta. La segunda: el ruido del juez no es aleatorio, se concentra en contextos largos y en fallos de parser, así que un timeout más generoso y menos concurrencia lo reducen.

El acuerdo humano también delimita lo que se puede afirmar. La corrección (0,95) y las citas (0,90) son fiables; la abstención (0,72) es aceptable; la fidelidad (0,12) no. Usar la fidelidad para elegir modelo habría sido elegir con ruido. La rúbrica de fidelidad mezcla errores factuales con abstenciones injustificadas, y los anotadores no comparten el criterio. Un eje que no alcanza acuerdo no debe decidir, y eso es una lección de diseño de la evaluación, no un defecto de los anotadores.

IX-E. Lo que mostró el conjunto reservado

El conjunto reservado cambia el diagnóstico: la caída de recuperación se confirma fuera del piloto y la unión con BM25 recupera parte de lo perdido. La mejora no viene de reordenar más: viene de recuperar entidades que el denso no trae. Quedan tres fallos de respuesta en la corrida sin compuerta: r-006 se abstiene, r-012 enumera iniciativas y r-016 es ambigua. El sistema sabe decir que no, pero todavía no siempre elige la evidencia correcta cuando hay varias.

La compuerta anterior tampoco generalizaba el cero. La nueva reduce los falsos rechazos, pero no los elimina; los que quedan son preguntas que la recuperación híbrida sí puede responder, porque la compuerta solo ve el top-5 denso. La lección no es retirar la compuerta, sino alinearla con el conjunto que ve el generador.

El conjunto reservado también muestra que el silver exploratorio no era representativo en idioma: el inglés, que parecía el punto débil, recupera mejor en la muestra nueva. Con 8 y 12 preguntas, las diferencias no son concluyentes, pero bastan para no tratar el idioma como una conclusión cerrada.

IX-F. Qué se llevaría a producción

La configuración que se llevaría es la de la Tabla V: unión densa y BM25, reranker en GPU, $k = 10$, gemma4:e4b con MTP y compuerta con umbral 0,4. Las deudas son r-006, r-016, la alineación de la compuerta con el conjunto reordenado y la corrección de la respuesta.

El monitoreo que acompañaría el despliegue tiene cuatro señales: recall en una muestra fija del silver, tasa de abstención, tasa de citas válidas y latencia p95. Las cuatro se pueden calcular con la infraestructura actual y detectan las regresiones que importan: que el índice cambie, que el modelo se actualice, que el prompt se rompa o que el hardware se quede sin memoria. La compuerta se monitorea aparte, con un conjunto fijo de preguntas de fuera, porque su deriva no se ve en las métricas de respuesta.

X. Ambigüedad y clarificación

X-A. Estado del arte

La ambigüedad referencial aparece cuando una pregunta admite varias respuestas válidas sobre el mismo corpus. AmbigQA la formaliza como una pregunta con un conjunto de respuestas [23]. CLAM muestra que los modelos detectan la ambigüedad cuando se les pregunta de forma explícita, pero en modo QA contestan igual; el contexto recuperado reduce aún más la probabilidad de pedir una aclaración [24]. CLAMBER evalúa la detección y la calidad de las preguntas de clarificación en cuatro tipos de ambigüedad y encuentra que los LLM actuales tienen dificultades para resolverla [25]. Clarify When Necessary estima la entropía sobre las intenciones para decidir cuándo preguntar [26].

En RAG hay dos familias. La clarificación interactiva pregunta al usuario y reescribe la consulta; ASK elige entre clarificación basada en documentos y basada en aspectos según la ambigüedad [27], y Tree of Clarifications construye un árbol de subpreguntas y agrega las respuestas [28]. La respuesta condicional contesta todas las interpretaciones; DIVA recupera pasajes diversos, verifica su calidad y adapta la respuesta [29], y CondAmbigQA introduce condiciones explícitas y reporta una mejora de 11,75 % cuando el modelo las considera [30]. La encuesta de 2025 ordena estas estrategias y señala que la detección sigue siendo el cuello de botella [31].

X-B. Diseño

El sistema usa un detector selectivo. Solo gasta una llamada al LLM cuando la pregunta contiene una referencia genérica que puede tener varios referentes en el corpus (“la iniciativa”, “el concierto”, “la reforma”, “el acuerdo”, “la COP”, entre otras). Para esas preguntas recupera una unión profunda de 50 fragmentos densos y 50 de BM25, la reordena y pasa el top-10 al detector. Así las opciones no se limitan al top-10 de producción.

El detector pide al LLM un JSON con la ambigüedad, la pregunta de clarificación y entre dos y seis opciones; cada opción lleva etiqueta, consulta reescrita, respuesta y evidencias. Una aumentación determinista añade las cifras de votos y los montos que el modelo omite, para que la lista cubra interpretaciones que el LLM no propuso. La clarificación devuelve las opciones a la interfaz; al elegir una, la consulta reescrita se envía con la compuerta desactivada, porque la interpretación ya está fundamentada. La respuesta condicional usa las mismas opciones como condiciones, cada una con su cita. La política híbrida clarifica en `/chat` y condiciona en `/ask`.

Tabla XXXI
Ambigüedad: 6 ambiguas y 6 de control. Baseline y clarificación: cobertura de tokens; condicional: resultado de la política sobre 12. Cobertura entre opciones: 5/6. La híbrida reporta clarificación interactiva y condicional de una pasada.

Política	Resultado	Turnos	Latencia
Baseline	0,833	1,0	7,16 s
Clarificación	0,833	1,4	8,78 s
Condicional	**0,917**	**1,0**	5,94 s
Híbrida	0,833 / 0,917	1 a 2	5,94 a 8,78 s

X-C. Experimento

El conjunto tiene 12 preguntas: 6 ambiguas y 6 de control, con la interpretación pretendida, las opciones, las respuestas condicionales y los alias en español e inglés. Se congeló antes de ejecutar. Se comparan cuatro políticas: baseline (respuesta única), clarificación, condicional e híbrida. La clarificación se evalúa con un usuario oracle que elige la opción cuya respuesta coincide con la interpretación pretendida; si no existe, se marca como fallo. Los valores p son de McNemar exacto bilateral y no se ajustan por comparaciones múltiples. La Tabla XXXI resume.

X-D. Resultados

El detector marcó las 6 ambiguas y ninguna de las 6 de control: precisión, recall y F1 de 1,0. En un segundo conjunto de 6 preguntas temporales y de alcance (3 ambiguas y 3 de control, estos últimos repetidos del primero) volvió a marcar las 3 ambiguas y ninguna de las 3 de control, con F1 de 1,0. Los dos conjuntos suman 15 preguntas únicas, incluidas en el v2 de 42. Los conjuntos son pequeños y el resultado es descriptivo, no una estimación poblacional.

El baseline cubre los tokens de referencia en 10 de 12 (0,833); la clarificación en 10 de 12 (0,833), con la interpretación pretendida en 5 de las 6 ambiguas; el resultado de la política condicional es 11 de 12 (0,917); la cobertura entre opciones es 5 de 6 (0,833). El condicional obtiene el mayor resultado automático y la menor latencia; la clarificación gana en control. La híbrida no gana en una métrica única.

En el conjunto temporal y de alcance, las tres políticas acertaron las 3 ambiguas y el condicional fue el más rápido (5,8 s). La recomendación por puntuación de la evidencia acertó 1 de 3 y no es un desambiguador fiable: el valor del condicional está en la cobertura, no en la recomendación, que se muestra solo como pista.

El conjunto v2 tiene 42 preguntas creadas por los autores; el silver asistido las etiquetó con un LLM y la doble anotación humana revisó la etiqueta de ambigüedad. Dejó 21 ambiguas y 21 de control (42/42, kappa 1,000). Cuatro preguntas que el silver asistido marcaba como ambiguas quedaron como control; dos tienen respuesta verificada (v-008 y v-009) y dos no (la tarifa del pasaje y el número de votos), así que la exactitud se mide sobre 40. El detector da F1 0,894 (precisión 0,81; recall 1,0); los cinco falsos positivos son esas cuatro y el proyecto Triángulo del Puma. El baseline cubre los tokens de referencia en 30 de 40 (0,750); la clarificación en 29 de 40 (0,725, p=1,0); el resultado de la política condicional es 33 de

40 (0,825), con cuatro pares a favor y uno en contra (McNemar p=0,375); la cobertura entre opciones es 18 de 21 (0,857). Sobre el silver asistido sube a 36 de 42 (p=0,039); la cobertura entre opciones es 22 de 25 (0,88). Los valores p comparan el resultado automático, no una evaluación semántica. Por tipo, acierta 1,0 en alcance, comparación, conversación y unidad, 0,78 en referencial y 0,75 en temporal. La recomendación se retira.

La lista plana tiene un límite. En la pregunta de los votos, el LLM propuso tres iniciativas y ninguna era la de 95 votos; la aumentación determinista añadió las cifras de la evidencia y la interpretación del silver entró como sexta opción. En el terremoto, la aumentación se contaminó con montos y unidades de otros documentos; el filtro por la unidad de la pregunta dejó las tres cifras de municipios y la de 472 entró como cuarta. Para preguntas con muchas interpretaciones, una clarificación jerárquica, que primero pregunte la faceta y después la opción, es el siguiente paso.

XI. Limitaciones

1. **Texto sin cierre.** El 32,7 % de los documentos de la muestra no termina en un signo de cierre. Es un proxy de truncamiento, no una verificación de que el rastreador los cortó: una publicación completa puede acabar sin punto. La limpieza no puede recuperar ese texto y ningún recuperador puede encontrar lo que no está.
2. **Sesgo de fuentes.** Cuatro cabeceras concentran la prensa y Bluesky llega casi todo en inglés. El corpus no representa la conversación pública en español ni la diversidad de medios del país.
3. **Fechas incompletas.** El rango declarado va de 2008 a 2026, pero la distribución utilizable se concentra en 2024 a 2026. El filtro temporal solo es fiable en ese tramo.
4. **Silver pequeño y desbalanceado.** 43 preguntas, 5 sin respuesta y 24 en prueba; 36 son de prensa y 2 de Bluesky, frente a un corpus con 38,2 % de documentos de Bluesky. Los intervalos de confianza son amplios y las diferencias de una o dos preguntas no son concluyentes. La revisión de Erika está registrada; algunas correcciones las aplicó un solo revisor y las filas marcadas sin reemplazo siguen abiertas.
5. **Conjunto reservado pequeño.** 20 directas y 5 negativas. Una sola pregunta mueve 5 puntos porcentuales y los intervalos son amplios. La recuperación híbrida se eligió sobre este conjunto, así que sus métricas de recuperación y respuesta son de desarrollo; la compuerta se midió antes de elegir el híbrido, pero el mismo conjunto se usó después para esa elección, así que sus cifras también son de desarrollo.
6. **Recall con un solo documento.** El silver reservado tiene un documento de referencia por pregunta. Si otro documento contiene la misma respuesta, el recall lo cuenta como fallo; r-005, r-008, r-017 y r-019 responden bien sin el documento de referencia. El recall@10 de 0,700 es una cota inferior de la recuperación útil.
7. **Compuerta no concluyente.** La compuerta nueva rechaza las 5 negativas del conjunto reservado y 3 de 20 directas; dos de esos tres son preguntas que la recuperación híbrida sí puede responder, pero la compuerta solo ve el top-5 denso. La tasa real sigue sin estimarse con precisión.
8. **Ampliación sin revisión humana.** Las 30 preguntas de la ampliación se generaron desde documentos aleatorios y el span literal es el gold, pero el campo `validated` sigue en falso. Algunas preguntas son claras (el Parque Tayrona, la misión Aspides) y otras dependen de un contexto que el documento no fija. La exactitud de 0,800 es la estimación más honesta porque el conjunto no participó en ninguna decisión, pero su gold no está verificado y la muestra es pequeña.
9. **Ambigüedad con 15 preguntas únicas y 42 anotadas.** El v2 tiene 21 ambiguas y 21 de control. Las 15 son preguntas diseñadas por los autores; los conjuntos de 12 y 6 comparten tres controles. La doble anotación dio 42 de 42 etiquetas idénticas (kappa 1,000), pero el kappa mide la coincidencia en la etiqueta binaria de ambigüedad sobre 42 casos; no valida la corrección de las opciones ni la representatividad poblacional. Dos controles no tienen respuesta verificable en el corpus (la tarifa del pasaje y el número de votos) y se excluyen de la exactitud (n=40). El detector deja cinco falsos positivos y el condensador conserva el sujeto anterior en preguntas encadenadas, lo que hunde v-008. La conversación queda en 1 de 1 evaluable. La lista plana y la jerarquía siguen pendientes; la recomendación se retiró.
10. **Ruido del juez.** RAGAS depende de un modelo local que falla en algunas muestras y cambia con el timeout y la concurrencia. La comparación con la arquitectura anterior cambia juez y embedding, así que no es completamente equivalente. El artefacto mide la configuración densa con $k = 5$ y sin compuerta estricta, antes de incorporar el híbrido.
11. **Hardware e infraestructura.** La estación tiene 16 GB de VRAM y 31 GB de RAM. El contexto de 116K ocupa 13,9 GB; el reordenador en GPU añade 1,6 GB y el margen es estrecho. La capa de datos (etcd, MinIO, Milvus y Solr) corre como stack de Docker Swarm en un servidor de la red local y el API la alcanza por LAN en el puerto 19530, sin túnel. La aplicación con GPU sigue en Compose porque Swarm no pasa dispositivos al contenedor.
12. **Alcance.** El sistema está diseñado para un corpus concreto de prensa y redes. No es un RAG de propósito general y las decisiones de limpieza, chunking y compuerta no se transfieren sin medir.

XII. Conclusiones y trabajo futuro

En el silver exploratorio el sistema responde 37 de 38 preguntas respondibles y se abstiene en las 5 sin respuesta. De las 37, 25 cubren los tokens de la respuesta de referencia; de las 12 restantes, la revisión manual de una persona juzgó correctas 8, parciales 3 e incorrecta 1. En el conjunto reservado entrega 19 de 20, cubre los tokens de referencia en 17 y se

abstiene en las 5; con la compuerta activa entrega 15 de 20. En conversación alcanza 14 de 14 con compuerta y 11 de 14 sin ella.

La primera lección es metodológica: el piloto sobreestima la recuperación. Con el mismo codificador y las mismas preguntas, Recall@10 cae de 0,947 a 0,684 al pasar de 1.932 a 161.868 documentos. Una comparación de codificadores o de chunking hecha solo sobre una muestra pequeña no permite elegir con confianza. El conjunto reservado confirma la caída. La unión con BM25 y el reordenamiento recuperan parte de ella, pero no la cierran. La segunda es que el orden y la cobertura se atacan en sitios distintos. Reordenar el top-5 sube el nDCG 5,6 puntos y el P@1 13,1 sin perder Recall@10, pero no recupera lo que falta; la unión con BM25 sí recupera entidades que el denso no trae. La profundidad de la unión tampoco es monotónica: 50/50 recupera más documentos de referencia, pero baja la cobertura de tokens; 10/20 es el mejor equilibrio. La tercera es que la limpieza conservadora basta: ninguna variante supera a otra de forma significativa, y la adopción se apoya en la consistencia y en eliminar páginas de error, no en una mejora demostrada de la normalización. El backend de recuperación es sustituible: el mismo contrato corre sobre Milvus o Solr. Solr iguala el recall del BM25 de Milvus; con bge-m3 y el índice HNSW por defecto se queda por debajo del denso de Milvus, así que la elección depende de la cobertura y la latencia, no solo del motor.

La cuarta lección es que la abstención barata es la compuerta. Rechazar antes de generar cuesta decenas de milisegundos y evita que el modelo improvise sobre temas ajenos. La quinta es que la evaluación automática necesita cobertura: una media de RAGAS con 4 valores de 38 invirtió una conclusión sobre la precisión del contexto. Reportar n junto a cada media no es un detalle de estilo; cambia lo que se concluye. La sexta lección es que, en estas muestras, el resultado automático del condicional es mayor que el del baseline: 33 de 40 frente a 30 de 40 en el silver humano, con cuatro pares a favor y uno en contra (McNemar p=0,375); la cobertura entre opciones es 18 de 21. La diferencia no alcanza significancia con 40 casos. En el silver asistido, 36 de 42 (p=0,039) y 22 de 25 de cobertura. La clarificación no se separa del baseline (29 de 40, p=1,0) y el detector baja a F1 0,894 por cinco falsos positivos. La recomendación por evidencia se retira. La clarificación interactiva da control pero cuesta un turno y una llamada. La política híbrida clarifica en la interfaz y condiciona en la API de una pasada. La séptima lección es que el silver exploratorio no basta. El conjunto reservado muestra el límite de la compuerta anterior. La compuerta con cross-encoder rechaza las 5 negativas y reduce los falsos rechazos, pero no los elimina; dos de los tres que quedan son preguntas que la recuperación híbrida sí puede responder. Un conjunto de 5 negativas no distingue una tasa nula de una tasa baja.

La octava lección es que la observabilidad cambia el tipo de afirmación que se puede hacer. Con Langfuse, cada consulta deja una traza con los fragmentos, el prompt, la respuesta, las citas y los tokens, y cada corrida se convierte en un experimento que enfrenta la respuesta esperada con la del modelo. Las tres corridas del silver comparten modelo, temperatura y conjunto, así que la diferencia es atribuible al recuperador: 0,837 de exactitud para la unión híbrida, 0,861 para el grafo híbrido y 0,861 para el boost y 0,884 para la configuración final, con la cobertura de tokens subiendo de 0,626 a 0,664. La novena es que GraphRAG no es una mejora gratuita. El grafo como fuente de candidatos sube una pregunta en el silver pero pierde 20 puntos en el conjunto multi-documento; el grafo como reordenador sube la cobertura y el anclaje sin degradar la exactitud. Con una extracción de entidades sin enlace canónico, el uso defendible es el boost. La décima lección es que el prompt es una palanca de calidad tan importante como el índice: con el mismo recuperador, el prompt v3 sube la corrección RAGAS de 0,590 a 0,652 y mantiene 0,917 de exactitud en test; sin el boost llega a 0,687 a costa de una pregunta. La diferencia pareada no excluye cero con 19 preguntas (IC 95 % $[-0{,}019, 0{,}145]$, $p = 0{,}14$), así que la mejora es una tendencia, no un resultado concluyente. La exactitud léxica subestima las respuestas correctas en otro idioma: dos casos del reservado (*World Economic Forum* y *Asian Hornet*) son correctos y el chequeo de contención los marca como fallo, mientras RAGAS los puntúa por encima de 0,7. La undécima lección es que la traducción y la extracción de listas atacan dos fallos concretos: g-046 (Newcastle) y g-003 (las cinco opciones). Con ambas, el test sube a 0,958 y el silver completo a 0,884; el reservado se queda en 0,840. La verificación numérica se probó y se descartó: en la corrida de test acertó g-044 y llegó a 1,000, pero en una consulta en vivo añadió 2017 en lugar de 2024 y RAGAS bajó de 0,668 a 0,655 en test y de 0,623 a 0,595 en el reservado; es un parche léxico inestable. Las diferencias no excluyen cero (IC 95 % $[-0{,}116, 0]$ en el silver), así que son tendencias. La duodécima lección es que la exactitud depende del conjunto: el test curado da 0,958 y la ampliación aleatoria de 30 preguntas baja a 0,800, mientras RAGAS se mantiene en 0,674 y la cobertura de contexto en 1,0. El test pequeño y elegido a mano infla la exactitud. El trabajo futuro tiene seis frentes. Primero, cerrar los fallos que quedan: g-044 (el año), r-006 (la cifra de la cámara) y r-016 (pregunta ambigua); la traducción resolvió g-046 y la extracción de listas g-003. La recuperación híbrida ya había resuelto r-012 y r-014. Segundo, alinear la compuerta con el conjunto reordenado, porque hoy rechaza dos preguntas que el generador podría responder. Tercero, mejorar la corrección de la respuesta, la métrica más débil de RAGAS. Cuarto, ampliar el conjunto de prueba para reducir los intervalos de confianza. Quinto, probar un chunking distinto y una reindexación completa, que es la única vía que queda para atacar la pérdida de contexto sin sacrificar cobertura. Sexto, probar una clarificación jerárquica para preguntas con muchas interpretaciones, porque la lista plana no las cubre todas.

XIII. Reproducibilidad

XIII-A. Hardware y software

El sistema se ejecutó en una estación con Ryzen 9 5950X, 31 GB de RAM y una GPU AMD Radeon RX 9070 XT de 16 GB con ROCm 7.2.4. La capa de datos (etcd, MinIO y Milvus 2.6.6) corre en un servidor de la red local con 16 núcleos y 62 GB de RAM; el API, el embedding y el

Tabla XXXII
MODELOS, ETIQUETA Y PAPEL EN EL SISTEMA.

Papel	Etiqueta	Cuantización
Generador	`gemma4:e4b`	Q4_K_M
Condensador y juez	`gemma2:2b`	Q4_K_M
Embedding	`BAAI/bge-m3`	fp16 en GPU
Reordenador	`BAAI/bge-reranker-v2-m3`	fp16 en GPU
Comparación E4	`qwen2.5:1.5b`	Q4_K_M
Comparación E4	`llama3.2:3b`, `llama3.2:1b`	Q4_K_M
Comparación E3	`qwen3-embedding:8b`	Q4_K_M

generador siguen en la estación. El API se conecta a Milvus por LAN en `http://192.168.1.100:19530`; el stack de datos se despliega con Docker Swarm y la aplicación con Compose. Los contenedores usan Python 3.12, PyTorch 2.10 con ROCm, FastAPI, Streamlit y Ollama. RAGAS 0.4.3 y LangChain 0.3 corren en un contenedor aparte con Python 3.12 y `BAAI/bge-m3` en CPU.

XIII-B. Modelos

La Tabla XXXII fija la versión de cada modelo. Los identificadores son las etiquetas de Ollama y de Hugging Face usadas en el código; no se usan alias móviles.

El generador, el detector y el condensador corren con temperatura 0; las repeticiones del mismo estado dan las mismas métricas. El artículo se compila con IEEEtran V1.8b. La corrida de referencia de la configuración densa es el commit `125bbc7`; la de la configuración híbrida es `experimento_recuperacion_hibrida.py`. Las figuras y tablas se regeneran desde `reports/tablas/`. RAGAS 0.4.3 corre con LangChain 0.3 y `BAAI/bge-m3` en CPU.

XIII-C. Comandos

El stack completo se levanta con un comando:

```
make all-up
make ps
curl http://localhost:8010/health
```

`make all-up` construye y arranca el stack completo (etcd, MinIO, Milvus, Solr, Ollama, API y frontend), espera a que la API esté sana, descarga `gemma4:e4b` y `gemma2:2b`, ingiere el Parquet, construye la colección dispersa e indexa Solr en texto y vectores. El servicio `prewarm` carga el generador, el embedding y el reordenador cuando la API está sana. El apagado deja los volúmenes intactos:

```
make all-down
```

La capa de datos del servidor se despliega con el stack de Swarm y el core de Solr con su esquema:

```
scripts/deploy_solr.sh
```

El grafo se construye desde el Parquet con `make graph-build`, que ejecuta `build_graph` por lotes de 1000. La evaluación se corre contra un conjunto y se registra como experimento de Langfuse:

```
make eval-langfuse RUN=extra-30
```

El conjunto y el destino se pasan con los flags del script:

```
python -m src.scripts.eval_langfuse \
  --silver eval/silver/silver_extra.jsonl \
  --dataset silver-extra --run extra-30
```

La configuración desplegada se fija en `.env`:

```
RETRIEVER=hybrid_boost
PROMPT_VERSION=v3
TRANSLATION=1
LIST_EXTRACTION=1
NUMERIC_VERIFICATION=0
QUERY_EXPANSION=0
CANDIDATE_GUARANTEE=0
```

XIII-D. Demo pública

La aplicación quedó publicada en https://tallerrag.hardwareelite.online mediante un túnel de Cloudflare. El túnel `taller-rag` se creó con la API de Cloudflare, con configuración remota y un solo ingress: el hostname público apunta a `http://localhost:8501`, el puerto del frontend, y el resto de las rutas devuelve 404. El DNS es un CNAME proxied a `<id>.cfargotunnel.com`. El conector corre como un contenedor más (`docker-compose.tunnel.yml`, red de host y `restart: unless-stopped`) y lee el token de `docker/cloudflared/.env`, que no se versiona. La página no tiene autenticación: es una demo abierta del taller, no un despliegue de producción.

La aplicación corre en la estación y el frontend habla con el API por `localhost:8010`; el API usa el Milvus y el Solr del servidor (`192.168.1.100`) y el generador y el embedding locales. Cloudflare publica solo el 8501, de modo que la API no queda expuesta. La Figura 16 muestra una consulta real hecha por la URL pública.

La verificación fue: `curl` al hostname devuelve 200 con el HTML de Streamlit, el túnel registra cuatro conexiones QUIC y un navegador headless completa la pregunta y recibe la respuesta con citas. La primera consulta por la URL pública tarda 10,5 s porque carga el reordenador; con el modelo caliente, la misma pregunta tarda 4,5 s (60 ms de recuperación y el resto de generación).

XIII-E. Observabilidad y grafo

El stack de observabilidad se despliega en el servidor y se detiene sin borrar volúmenes:

```
make obs-up
make obs-down
```

`make obs-up` usa el compose de observabilidad con su archivo de entorno no versionado. Las capturas de Langfuse y Neo4j se generan con `scripts/capture_firefox.py` y Firefox headless. El modo del recuperador se elige por petición con el campo `retriever` (`hybrid_union`, `hybrid_boost`, `graph_hybrid`, `solr`).

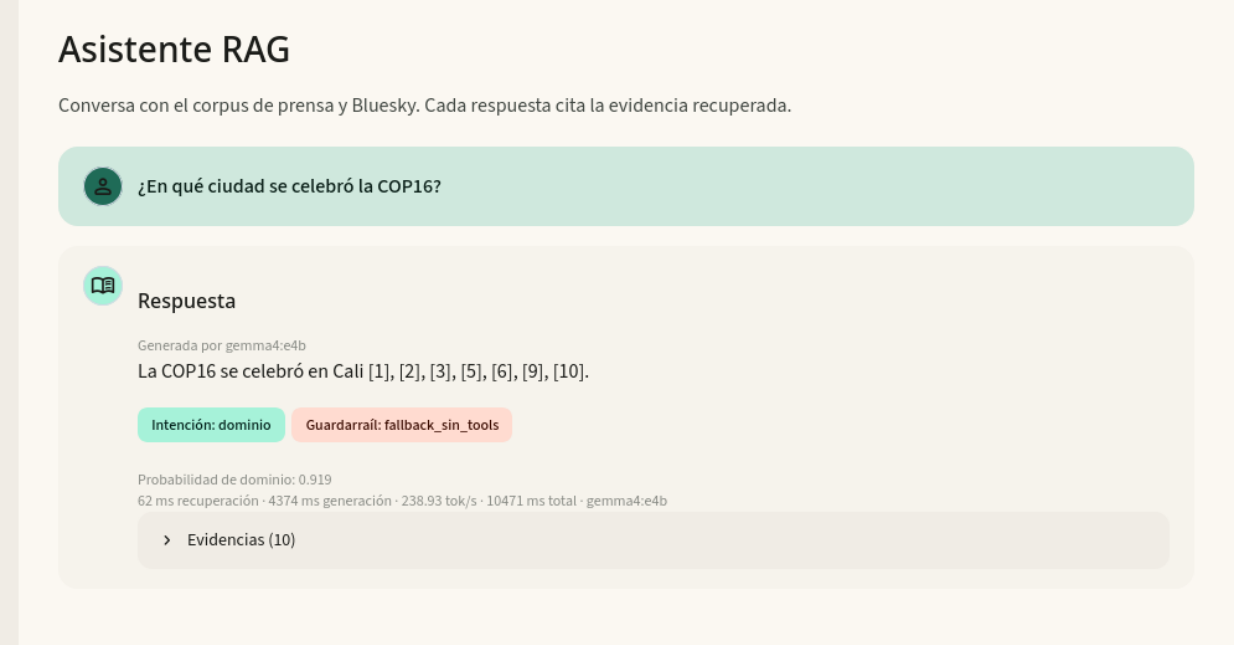


Figura 16. Demo pública: consulta real por `tallerrag.hardwareelite.online`. La respuesta cita siete de las diez evidencias recuperadas, la compuerta acepta la pregunta con probabilidad 0,919 y la interfaz muestra la ruta y la latencia. La primera consulta paga la carga del reordenador; en caliente, la misma pregunta tarda 4,5 s.

XIII-F. Artefactos

El silver consolidado está en `eval/silver/silver_standard.jsonl`, con su versión ampliada en `silver_standard_ampliado.jsonl`. El conjunto reservado (`silver_reservado.jsonl`, con hashes en `README_RESERVADO.md`) escribe `reports/corridas/reservado_*_raw.jsonl` y `reports/tablas/reservado_*_resumen.csv`. El silver de ambigüedad está en `eval/silver/silver_ambiguedad*.jsonl`, con las tablas `ambiguedad_*_resumen.csv` y `ambiguedad_*_por_pregunta.csv`. La ampliación de 30 preguntas está en `eval/silver/silver_extra.jsonl`, su corrida en `reports/corridas/eval_langfuse_extra-30.jsonl` y su tabla RAGAS en `reports/tablas/ragas_extra.csv`. La capa de datos se define en `docker-stack.yml`, el stack completo en `docker-compose.all.yml` y el modo local de respaldo en `docker-compose.data.yml`. Las figuras están en `reports/figuras/`, las corridas en `reports/corridas/` y los modelos de la compuerta en `modelos/`.

XIII-G. Datos

El corpus completo no se publica por derechos de las fuentes. Se incluyen el silver, las anotaciones, la muestra del piloto y los Parquet de evaluación necesarios para reproducir las métricas. Los Parquet se versionan con Git LFS (`*.parquet filter=lfs`); después de clonar hay que ejecutar `git lfs pull` para materializarlos. Las cifras de este artículo se pueden verificar contra esos artefactos sin volver a descargar el corpus.

Referencias

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in *NeurIPS Datasets and Benchmarks Track*, 2021, arXiv:2104.08663.

[3] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive text embedding benchmark," in *Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2023, arXiv:2210.07316.

[4] X. Zhang, N. Thakur, O. Ogundepo, E. Kamalloo, D. Alfonso-Hermelo, X. Li, Q. Liu, M. Rezagholizadeh, and J. Lin, "MIRACL: A multilingual retrieval dataset covering 18 diverse languages," *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 1114–1131, 2023, arXiv:2210.09984.

[5] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," *arXiv preprint arXiv:2309.15217*, 2023.

[6] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "ARES: An automated evaluation framework for retrieval-augmented generation systems," in *North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024, arXiv:2311.09476.

[7] A. Salemi and H. Zamani, "Evaluating retrieval quality in retrieval-augmented generation," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024, arXiv:2404.13781.

[8] D. Ru, L. Qiu, X. Hu, T. Zhang, P. Shi, S. Chang, C. Jiayang, C. Wang, S. Sun, P. Li *et al.*, "RAGChecker: A fine-grained framework for diagnosing retrieval-augmented generation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2408.08067.

[9] Zilliz, "Milvus documentation," https://milvus.io/docs, 2026.

[10] Ollama, "Ollama: Get up and running with large language models locally," https://github.com/ollama/ollama, 2026.

[11] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[12] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," in *Proceedings of the 32nd International ACM SIGIR Conference*, 2009, pp. 758–759.

[13] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-rag: Learning to retrieve, generate, and critique through self-reflection," *arXiv preprint arXiv:2310.11511*, 2023.

[14] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective retrieval augmented generation," *arXiv preprint arXiv:2401.15884*, 2024.

[15] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Multilingual E5 text embeddings: A technical report," *arXiv preprint arXiv:2402.05672*, 2024.

[16] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," *arXiv preprint arXiv:2402.03216*, 2024.

[17] R. Nogueira and K. Cho, "Passage re-ranking with bert," *arXiv preprint arXiv:1901.04085*, 2019.

[18] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[19] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024, arXiv:2307.03172.

[20] C. Niu, Y. Wu, J. Zhu, S. Xu, K. Shum, R. Zhong, J. Song, and T. Zhang, "Ragtruth: A hallucination corpus for developing trustworthy retrieval-augmented language models," *arXiv preprint arXiv:2401.00396*, 2024.

[21] S. Ramírez, "FastAPI documentation," https://fastapi.tiangolo.com, 2026.

[22] Streamlit Inc., "Streamlit documentation," https://docs.streamlit.io, 2026.

[23] S. Min, J. Michael, H. Hajishirzi, and L. Zettlemoyer, "AmbigQA: Answering ambiguous open-domain questions," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 5783–5797.

[24] L. Kuhn, Y. Gal, and S. Farquhar, "CLAM: Selective clarification for ambiguous questions with generative language models," *arXiv preprint arXiv:2212.07769*, 2022.

[25] T. Zhang, P. Qin, Y. Deng, C. Huang, W. Lei, J. Liu, D. Jin, H. Liang, and T.-S. Chua, "CLAMBER: A benchmark of identifying and clarifying ambiguous information needs in large language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 10 746–10 766.

[26] M. J. Zhang and E. Choi, "Clarify when necessary: Resolving ambiguity through interaction with lms," in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 5541–5558.

[27] R. Sahay, L. S. Tekumalla, P. Aggarwal, A. Jain, and A. Saladi, "ASK: Aspects and retrieval based hybrid clarification in task oriented dialogue systems," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), Industry Track*, 2025, pp. 881–895.

[28] G. Kim, S. Kim, B. Jeon, J. Park, and J. Kang, "Tree of clarifications: Answering ambiguous questions with retrieval-augmented large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[29] Y. In, S. Kim, R. A. Rossi, M. M. Tanjim, T. Yu, R. Sinha, and C. Park, "Diversify-verify-adapt: Efficient and robust retrieval-augmented ambiguous question answering," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL)*, 2025, pp. 1212–1233.

[30] Z. Li, Y. Li, H. Xie, and S. J. Qin, "CondAmbigQA: A benchmark and dataset for conditional ambiguous question answering," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025.

[31] M. Tanjim, Y. In, X. Chen, V. Bursztyn, R. A. Rossi, S. Kim, G.-J. Ren, V. Muppala, S. Jiang, Y. Kim, and C. Park, "Disambiguation in conversational question answering in the era of llms and agents: A survey," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 9537–9550.